01001011 0110
1101 0010 1110
0111 00101

CMP103 · Hafta 4 · Vize hazırlığı

Veri Nasıl Saklanır?

Sayılar, metin, görüntü, ses, video ve bellek adresleri

13 Ekim 2026 | Kaynak materyale dayalı öğretici çalışma notu

İçindekiler

Başlangıç 3

Bu not ne, nasıl kullanılmalı? . . . 3
Dosya kodları ve sayfa etiketleri . . . 3
Gereken ön bilgiler . . . 3
Dersin haritası . . . 4
Kutular ve etiketler ne anlama geliyor? . . . 4
Ders bilgileri ve yaptığım varsayımlar . . . 4
Çalışma planı önerisi . . . 5

1 Veriden Bite: Veri, Bilgi ve Kodlama 6

1.1 62, 71, 84, 95: Bu sayılar sana bir şey söylüyor mu? . . . 6
1.2 Bilgisayar hangi tür verilerle uğraşır? . . . 7
1.3 Dünyadan bitlere: kodlama ve kod çözme . . . 8
1.4 Bit, bayt, bit deseni: bitler kendi başına bir şey söyler mi? . . . 8
1.5 n bit ile kaç farklı şey anlatabilirim? . . . 9
Bağlantılar . . . 11
Bölüm 1 cevapları . . . 12

2 Sayı Sistemleri ve Dönüşümler 14

2.1 3125'teki "3" neden 3000 eder? . . . 14
2.2 Bütün tabanlar için tek formül . . . 15
2.3 İkilik sistem ve donanımın sevgisi . . . 16
2.4 Tabanlar tablosu: "on" sayısını dört dilde yazmak . . . 16
2.5 Onaltılık sistem: bitlerin kısaltması . . . 17
2.6 Dönüşümler: altı yön, üç yöntem . . . 18
2.7 Slayt 28–29: on dönüşüm, tam çözüm . . . 22
Bağlantılar . . . 23
Bölüm 2 cevapları . . . 24

3 Metnin Temsili: ASCII'den UTF-8'e 25

3.1 Bir dosya "BYTE" kelimesini nasıl saklar? . . . 25
3.2 ASCII: 128 karakterlik İngilizce alfabe . . . 26
3.3 8 bitlik kod sayfaları ve "Ã§" karmaşası . . . 27
3.4 EBCDIC: IBM'in kendi yolu . . . 28
3.5 Unicode: Dünyanın bütün karakterlerine tek numara . . . 28
3.6 UTF-8, UTF-16, UTF-32: Aynı numarayı farklı paketlemek . . . 30
Bağlantılar . . . 31
Bölüm 3 cevapları . . . 32

4 Görüntünün Temsili **34**
4.1 Bir resim nasıl sayılara dönüşür? . . . 34
4.2 Siyah-beyaz: piksel başına tek bit . . . 35
4.3 Gri tonlama ve renk derinliği . . . 35
4.4 RGB: Ekran renkleri ışıkla karıştırır . . . 36
4.5 Çözünürlük ve boyut hesabı . . . 36
4.6 Palet: Renk yerine "renk numarası" saklamak. . . . 37
4.7 Vektör grafik: Piksel yerine tarif saklamak . . . 38
Bağlantılar . . . 39
Bölüm 4 cevapları . . . 40

5 Video ve Ses **42**
5.1 Video: saniyede 30 fotoğraf . . . 42
5.2 Ses: Sürekli dalgadan sayılara . . . 43
Bağlantılar . . . 45
Bölüm 5 cevapları . . . 46

6 Bellek Adresleme **47**
6.1 Bellek: numaralı çekmeceler . . . 47
6.2 Bayt adreslemeli bellek ve değişkenler . . . 48
6.3 n adres biti → 2^n bayt . . . 49
6.4 Adresler neden onaltılık yazılır? . . . 51
Bağlantılar . . . 52
Bölüm 6 cevapları . . . 53

Konular Arası Bağlantılar **54**

Sınav Öncesi Hızlı Tekrar **56**
Temel tanımlar . . . 56
Formül listesi . . . 56
Ezber tabloları . . . 56
Dönüşüm yöntemleri . . . 57
Kritik ayrımlar . . . 57
En sık 8 hata. . . . 57
Sınav sabahı kontrol listesi . . . 57

Deneme Sınavı **59**
Bölüm A: Çoktan seçmeli . . . 59
Bölüm B: Klasik sorular . . . 60

Ayrıntılı Cevap Anahtarı **62**
Bölüm A . . . 62
Bölüm B . . . 63

Terim Sözlüğü (İngilizce–Türkçe) **64**

Hocaya Sorulacak Noktalar **67**

Kaynak Kapsamı Kontrol Tablosu **68**

Başlangıç

Bu not ne, nasıl kullanılmalı?

Bu kitap, CMP103 dersinin 4. haftasındaki "How Is Data Stored?" slaytlarını temel alan bir **öğrenme kaynağı**. Slaytların çevirisi değil: slaytların bir cümleyle geçtiği her şeyi (neden ikilik sistem? neden 4 bit = 1 onaltılık rakam? neden adres bitinde logaritmanın tavanı?) sıfırdan, adım adım anlatıyor ve seni sınava hazırlıyor.

Önerilen okuma sırası, kitabın sırasıyla aynı. Çünkü her bölüm bir öncekine yaslanıyor: "n bit ile 2^n farklı değer" fikrini (Bölüm 1) kavramadan renk derinliğini (Bölüm 4) ya da adres bitlerini (Bölüm 6) anlamak zor; onaltılık sistemi (Bölüm 2) bilmeden de ASCII tablosu, renk kodları ve bellek adresleri sadece anlamsız harf yığını gibi görünür.

Her bölümü şu döngüyle oku:

1. Anlatımı oku, **Örnek** kutularını kalemle kendin de çöz.
2. **Durup düşün** ve **Kendin çöz** kutularında dur; cevaba bakmadan önce kâğıda yaz.
3. Bölüm sonundaki **Bölüm cevapları** ile kontrol et.
4. Bölüm sonu kutularını (Sınavda dikkat, Nasıl soru gelir, Sık yapılan hatalar, Hızlı tekrar) sınavdan önceki tekrar için işaretle.

Dosya kodları ve sayfa etiketleri

Kod	Dosya	İçerik
D1	CMP103_04.pdf	Hafta 4 slaytları, 66 sayfa (Biruni Üniversitesi, Güz 2026)

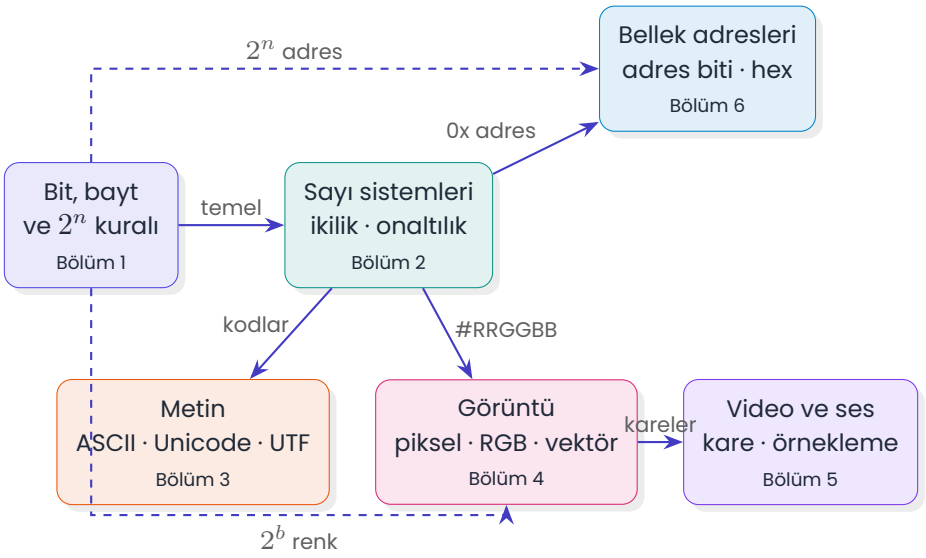
Etiketlerde **PDF görüntüleyicide gördüğün fiziksel sayfa numarası** kullanıldı. Slaytın köşesinde yazan numara farklıysa ikisi birlikte yazıldı: [D1 s. 31 / slayt 32]. 1–29. sayfalarda iki numara aynı; 30. sayfadan sonra slayt numaraları atlıyor (bazı slaytlar sunumdan çıkarılmış görünüyor), bu yüzden orada iki numarayı da görürsün.

Gereken ön bilgiler

Lise matematiğinden şunlar yeter; unuttuysan ilgili yerde **Hatırlatma** kutusu var:

- Üslü sayılar: $2^0 = 1$, $2^{10} = 1024$, $2^a \cdot 2^b = 2^{a+b}$ (Bölüm 1 ve 6)
- Bölme ve kalan (Bölüm 2)
- Logaritma: $\log_2 8 = 3$ demek "2'nin kaçıncı kuvveti 8?" sorusunun cevabı (Bölüm 6)
- Çarpma ve büyük sayılarla birim dönüştürme (Bölüm 4 ve 5)

Dersin haritası



Şekil 1: Haftanın kavram haritası: her şey "bit" ve "2^n" fikrinden doğuyor; onaltılık sistem de üç ayrı yerde (karakter kodları, renk kodları, adresler) karşına çıkıyor.

Kutular ve etiketler ne anlama geliyor?

Kutu / etiket	Anlamı
TANIM / FORMÜL (sarı)	Ezberlemen ya da türetebilmen gereken çekirdek bilgi
SINAVDA DİKKAT, NASIL SORU GELİR? (kırmızı)	Sınav odaklı uyarılar ve soru kalıpları
ÖRNEK (mavi)	Tam çözümlü örnek; "Slayt örneği" ya da "Ek örnek" diye etiketli
SIK YAPILAN HATA (turuncu)	Yanlış düşünce → doğrusu → neden
DURUP DÜŞÜN, KENDİN ÇÖZ (yeşil)	Önce sen dene; cevaplar bölüm sonunda
HATIRLATMA (gri-lacivert)	Lise bilgisi tazeleme
EK BİLGİ – KAYNAK DIŞI (mor, kesikli)	Slaytlarda olmayan ama anlaman için eklenen bilgi
MERAKLISINA – SINAV DIŞI (pembe)	Keyif için; sınavda sorulmaz
SINAV ÖNCELİĞİ: YÜKSEK SINAV ÖNCELİĞİ: ORTA SINAV ÖNCELİĞİ: DÜŞÜK	Sınav önceliği **tahmini**
❖ Kaynak: D1 s. 4–5	Bu kısmın dayandığı slayt sayfaları

Ders bilgileri ve yaptığım varsayımlar

Bilgi	Değer	Nereden
Ders	CMP103 – Introduction to Computer Engineering and Career Planning	D1 s. 1 kapak
Kapsam	Hafta 4: "How Is Data Stored?"	D1 s. 1
Sınav	Vize (varsayım)	4. hafta dönemin ilk yarısında

Bilgi	Değer	Nereden
Sınav tarihi	Bilinmiyor	→ esnek 7/3/1 gün planı
Sınav formatı	Bilinmiyor → karışık: çoktan seçmeli + klasik/hesap	Geçmiş yıl sorusu yüklenmedi
Sınav dili	Bilinmiyor → sorular İngilizce, altında Türkçe çeviri	Ders İngilizce
Programlama dili	Ders programlama dersi değil; s. 62'deki değişken örneği C/C++ türleriyle yazılmış	D1 s. 62
Vurgulanan yerler	Slaytların alt bandındaki koyu cümleler, s. 2'deki "Questions for this week", s. 3 öğrenme çıktıları	D1
Zor kabul edilenler	Dönüşümler, $\lceil \log_2 \rceil$ ile adres biti, boyut hesaplarında birim dönüşümleri, Unicode–UTF farkı	Adım ve hesap sayısı
Sayı yazımı	Slaytlarla aynı: ondalık ayırıcı **nokta** (1.37 MiB), binlik ayırıcı **virgül** (2,073,600). Türkçe yazımdan farklı, dikkat!	D1

Sınav öncelikleri ve "nasıl soru gelir" kutuları **benim tahminimdir**: hocanın gerçek sınavını görmedim. Gerekçeleri her bölümün başında yazıyor.

Çalışma planı önerisi

Ne zaman	Ne yap
Sınava 7 gün kala	Bölüm 1–2'yi oku, slayt 28–29'daki 10 dönüşümü kâğıtta çöz.
6–5 gün kala	Bölüm 3 ve 4. ASCII'de A=65, a=97, '0'=48 üçlüsünü ve görüntü boyutu formülünü oturt.
4 gün kala	Bölüm 5 ve 6. Adres biti tablosunu (s. 64) kendi kendine yeniden üret.
3 gün kala	Tüm "Kendin çöz" sorularını cevaba bakmadan yeniden çöz; yanlışlarını işaretle.
2 gün kala	Deneme sınavını süre tutarak (60 dk) çöz, cevap anahtarıyla puanla.
1 gün kala	"Sınav öncesi hızlı tekrar" bölümü + yanlış yaptığın soruların konuları.
Sınav sabahı	Sadece hızlı tekrar sayfaları ve 2'nin kuvvetleri tablosu.

BÖLÜM 1

Veriden Bite: Veri, Bilgi ve Kodlama

◆ **BU BÖLÜMDE** Okuma süresi: 35 dk Zorluk: ●○○

- Veri ile bilgi arasındaki farkı ve bağlamın rolünü
- Bilgisayarın işlediği beş veri türünü
- Kodlama (encoding) ve kod çözmenin (decoding) ne olduğunu
- Bit, bayt ve bit deseni kavramlarını; "bitlerin kendi başına anlamı yoktur" fikrini
- n bit ile neden tam olarak 2^n farklı değer gösterilebildiğini

SINAV ÖNCELİĞİ: ORTA ❖ Kaynak: D1 s. 4–10

Sınav önceliği orta: veri–bilgi ayrımı kısa tanım sorusu olarak gelebilir; ama **2^n kuralı yüksek öncelikli**, çünkü renk derinliği (Bölüm 4) ve adres bitleri (Bölüm 6) tamamen ona dayanıyor.

→ BU KONUYA NEDEN GEÇİYORUZ?

Bu haftanın tek büyük sorusu var: Telefonundaki bir fotoğraf, bir WhatsApp mesajı, bir şarkı... hepsi aynı çipin içinde nasıl duruyor? Cevabın ilk yarısı bu bölümde: önce "veri" ile "bilgi"yi ayırıyoruz, sonra her şeyin sonunda **0 ve 1'lere** dönüştüğünü ve bu 0–1'lerle kaç farklı şey anlatılabileceğini hesaplıyoruz. Bu bölümdeki 2^n kuralı, kitabın geri kalanının anahtarı.

1.1 62, 71, 84, 95: Bu sayılar sana bir şey söylüyor mu?

❖ Kaynak: D1 s. 4–5

Biri sana kâğıtta yalnızca "62, 71, 84, 95" yazıp verse ne anlarsın? Hiçbir şey. Sıcaklık mı, kilo mu, not mu belli değil. İşte bu çıplak değerlere **veri (data)** denir.

Şimdi aynı kâğıda "Bunlar dört öğrencinin vize notu, sınıf ortalaması 78" yazılsın. Artık bir soruya cevap veren, karar almanı sağlayan bir şey var: **bilgi (information)**.

∑ TANIM / FORMÜL — Veri ve bilgi

Veri (data): Gözlem, ölçüm ya da sembol biçimindeki ham değerler. Slayttaki tanım: *"Raw values in the form of observations, measurements, or symbols."*
Bilgi (information): Bir soruyu cevaplamak üzere yorumlanmış veri. Slayt: *"Data interpreted to answer a question."*
Veriyi bilgiye çeviren şey **bağlam (context) ve yorum (interpretation)**dur.

Slayttaki örnekte ortalama gerçekten 78: (62 + 71 + 84 + 95) / 4 = 312 / 4 = 78.

Tanımdaki anahtar kelimeler sınavda puan getirir: veri için **raw / ham**, bilgi için **interpreted / yorumlanmış** ve **answer a question / bir soruya cevap**.

▶ ÖRNEK 1.1 — Slayt örneği [D1 s. 5]: Sınıflandırma etkinliği

Soru: Her ifadeyi veri ya da bilgi olarak sınıflandır, bağlamın kararını nasıl etkilediğini açıkla.
Yöntem: Her kart için sor: "Bu ifade tek başına bir soruya cevap veriyor mu, yoksa ne olduğunu anlamak için ek bağlam mı gerekiyor?"

Kart	Karar	Gerekçe
35209852715	Veri	11 haneli çıplak bir sayı. TC kimlik no mu, telefon mu, sipariş no mu? Bağlam yok, yorum yok.
Sales today: 420	Çoğunlukla bilgi	"Bugünkü satış" etiketi bağlam ekliyor; "bugün kaç satış oldu?" sorusunu cevaplıyor. Ama "420 adet mi, 420 TL mi?" belirsiz; birimi bilmeyen biri için hâlâ eksik.
Best-selling newspaper	Bilgi	Satış verileri karşılaştırılıp yorumlanmış; "en çok satan hangisi?" sorusunun cevabı.
Assignment grade: 85	Bilgi (bağlama göre)	Öğrenci için "ödevden kaç aldım?" sorusunun cevabı. Ama sınıf ortalamasını hesaplayan hoca için 85, tek başına bir veri noktası.

Sonuç: Aynı değer, bağlam ve yorum eklendikçe veriden bilgiye geçer. Slaytın alt notu da tam bunu söylüyor: *"The same value can become information when context and interpretation are added."*
En sık hata: "Sayıysa veridir, cümleyse bilgidir" demek. Ölçüt biçim değil, **bir soruya yorumlanmış cevap olup olmadığı**.
Başka nerede işine yarar: Veritabanları, veri bilimi, yapay zekâ derslerinde "ham veri → işlenmiş bilgi" ayrımı hep karşına çıkacak.

✎ DURUP DÜŞÜN

1. "Bugün hava 23 °C" ifadesi senin için bilgi. Bir hava durumu istasyonunun her dakika kaydettiği 1440 sıcaklık ölçümü ise ne?
2. Aynı ifade hem veri hem bilgi olabilir mi? Bir örnekle açıkla.

1.2 Bilgisayar hangi tür verilerle uğraşır?

❖ Kaynak: D1 s. 6

Slayt beş veri türü sayıyor. Her birinin "doğal" hâli farklı:

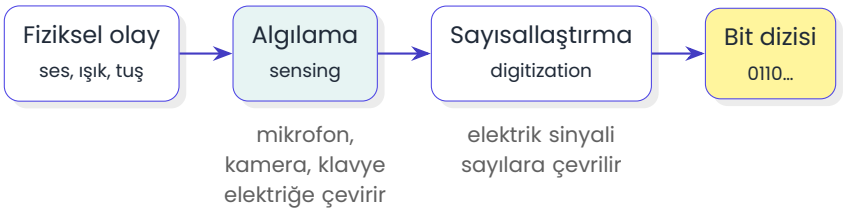
Tür	Doğal hâli (slayt)	Bu kitapta
Sayı (**number**)	ölçüm, sayım, hesap	Bölüm 2
Metin (**text**)	harf, rakam, noktalama, sembol	Bölüm 3
Görüntü (**image**)	pikseller ya da geometrik nesneler	Bölüm 4
Ses (**audio**)	zaman içinde örneklenen bir sinyal	Bölüm 5
Video (**video**)	zamanlanmış görüntü kareleri ve ses	Bölüm 5

Bu kadar farklı şeyin hepsi aynı belleğe sığıyor. Nasıl? Hepsi önce aynı dile, yani bitlere çevriliyor. Şimdi o çeviri sürecine bakalım.

1.3 Dünyadan bitlere: kodlama ve kod çözme

❖ Kaynak: D1 s. 7–8

Mikrofona konuştuğunda havada bir basınç dalgası oluşur; klavyede tuşa bastığında bir anahtar kapanır; kamera sensörüne ışık düşer. Bunların hiçbiri "0 ve 1" değil. Bilgisayara girebilmeleri için dört adımlık bir yolculuk gerekir:



Şekil 1.1: D1 s. 8'deki zincir. Metin, görüntü ve ses farklı yöntemlerle kodlanır ama hepsinin çıktısı bitlerdir.

∑ TANIM / FORMÜL — Kodlama ve kod çözme

Kodlama (encoding / coding): Veriyi bir bit dizisi olarak temsil etmek. *"Encoding represents data as a sequence of bits."*
Kod çözme (decoding): Bit dizisini kullanıcıya uygun biçime geri çevirmek. *"Decoding converts the bit sequence back into a form suitable for the user."*
Giriş aygıtı (**input device**) fiziksel sinyali elektriksel ya da sayısal biçime çevirir.

D1 s. 7'deki şemayı şöyle oku: soldaki beş veri türü sarı **Coding** kutularından geçip bit dizisi olur, hepsi aynı **Memory** kutusunda durur; sağda yeşil **Decoding** kutuları onları tekrar metin, sayı, görüntü... hâline getirir.

Benzetme: Kodlama, bir yemeği vakumlayıp dondurucuya koymak gibi. Dondurucu (bellek) içindekinin ne olduğuyla ilgilenmez, hepsi buz kalıbı gibi durur. Yemeği geri yiyebilmek için doğru "çözme" yöntemini (ısıtma talimatını) bilmen gerekir. **Benzetmenin bozulduğu yer:** Donmuş paketin üstünde genelde etiket olur; bellekteki bitlerin üstünde ise hiçbir etiket yoktur. Hangi kurala göre çözüleceğini bitler değil, onları okuyan program bilir. Bu, bir sonraki alt başlığın ana fikri.

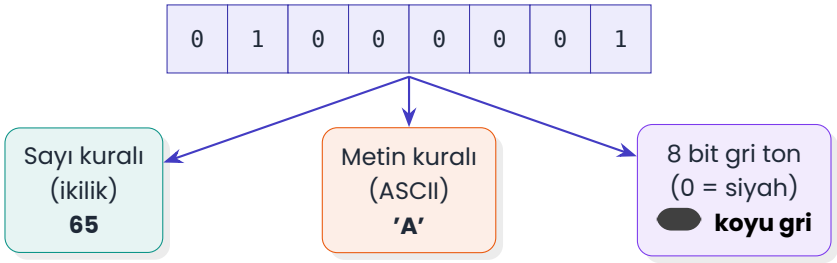
1.4 Bit, bayt, bit deseni: bitler kendi başına bir şey söyler mi?

❖ Kaynak: D1 s. 9

∑ TANIM / FORMÜL — Bit, bayt, bit deseni

- **Bit** (*binary digit*, ikilik rakam): En küçük sayısal veri birimi; değeri 0 ya da 1.
- **Bayt (byte):** 8 bit. *"Eight bits form one byte."*
- **Bit deseni (bit pattern):** Bir bit dizisi. Uzunluğu, içindeki bit sayısıdır (1011 deseninin uzunluğu 4).

Şimdi işin püf noktası geliyor. Bellekte şu 8 bit duruyor olsun: **01000001**. Bu ne?



Şekil 1.2: Aynı 8 bit, üç farklı kodlama kuralıyla üç farklı anlam taşır. 65/255 parlaklık, ölçeğin dörtte biri civarı olduğu için koyu bir gridir.

Slaytın alt bandındaki cümle bu yüzden önemli: *"Bits do not carry meaning by themselves. The encoding rule defines the meaning."* Yani bitlerin anlamını **kodlama kuralı** belirler.

★ MERAKLISINA (SINAV DIŞI)

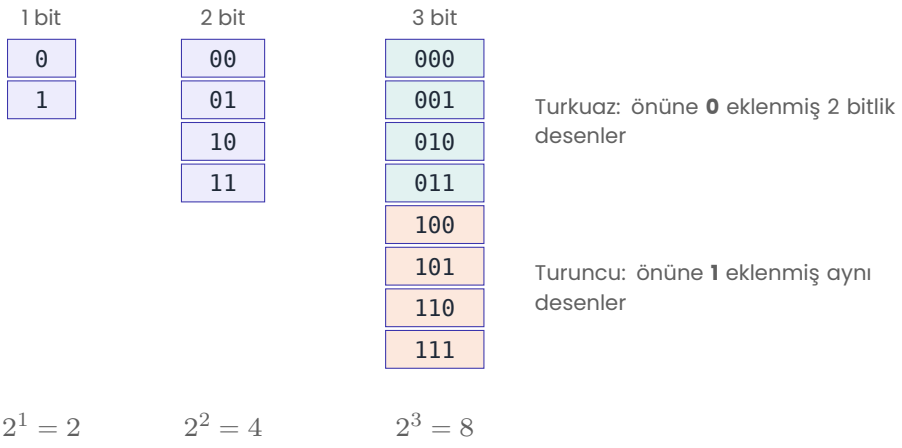
Bir `.txt` dosyasının adını `.jpg` yapıp açmayı denersen "dosya bozuk" hatası alırsın. Dosyanın içindeki bitler hiç değişmedi; değişen, program onları hangi kurala göre okumaya çalıştığı. Bilgisayarlar dosya türünü çoğu zaman uzantıdan ve dosyanın başındaki birkaç "imza" bayttan tahmin eder.

1.5 n bit ile kaç farklı şey anlatabilirim?

❖ Kaynak: D1 s. 10

Önce problem: Okulda 30 kişilik bir sınıf var ve her öğrenciye bitlerden oluşan bir numara vermek istiyorsun. Kaç bit yeter? Bunu cevaplamak için "n bit kaç farklı desen üretir?" sorusunu çözmeliyiz.

Küçükten başlayalım:



$2^1 = 2$ $2^2 = 4$ $2^3 = 8$

Şekil 1.3: Her yeni bit, mevcut desenlerin tamamını iki kopyaya ayırır: biri 0 ile, biri 1 ile başlayan. Bu yüzden desen sayısı her bitte ikiye katlanır.

Mantık şu: n bitlik desenleri listelediğini düşün. Başlarına bir bit daha eklediğinde, o bit ya 0 ya 1 olabilir. Her eski desenden iki yeni desen çıkar, toplam sayı ikiye katlanır. 1 bitte 2 desen varsa, her bitte ikiyle çarpa çarpa n bitte $2 \cdot 2 \cdot \ldots \cdot 2$ (n tane) = $\mathbf{2^n}$ desen olur.

Bunu bir de "sayma ilkesi" ile söyleyelim: Her bit pozisyonu için 2 seçenek var ve seçimler birbirinden bağımsız. n pozisyon için $2 \times 2 \times \ldots \times 2 = 2^n$.

∑ TANIM / FORMÜL — n bit → 2^n değer

n bitlik desenlerle tam olarak $\mathbf{2^n}$ farklı değer gösterilebilir.

n	1	4	8	16
2^n	2	16	256	65,536

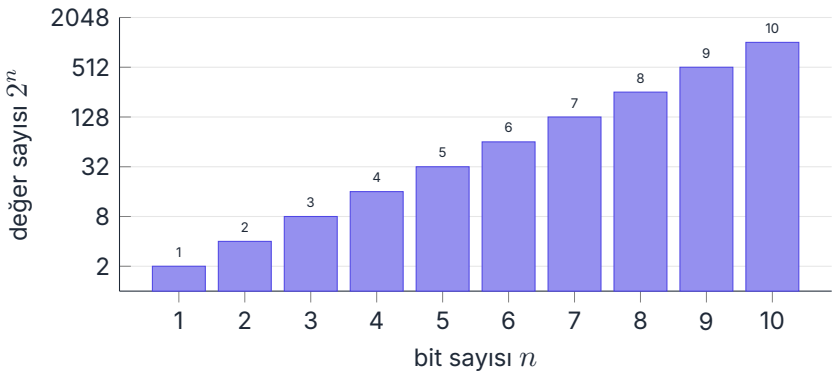
Her ek bit, gösterilebilecek değer sayısını **ikiye katlar** (*"Each additional bit doubles the number of values that can be represented."*).
Bu desenler 0'dan başlayan sayılara karşılık geliyorsa en büyük değer $\mathbf{2^n - 1}$'dir: 8 bitle 0…255.

↺ HATIRLATMA — İkinin kuvvetleri

Bu tabloyu ezberlemen hayatını kolaylaştırır (her biri bir öncekinin iki katı):

2^0	2^1	2^2	2^3	2^4	2^5	2^6	2^7	2^8	2^9	2^{10}
1	2	4	8	16	32	64	128	256	512	1024

Kural: $2^a \cdot 2^b = 2^{a+b}$. Örneğin $2^{16} = 2^8 \cdot 2^8 = 256 \cdot 256 = 65{,}536$ ve $2^{20} = 2^{10} \cdot 2^{10} \approx$ bir milyon.



Şekil 1.4: Logaritmik eksende 2^n: her çubuk bir öncekinin tam iki katı. 10 bitte zaten 1024 farklı değer var.

▶ ÖRNEK 1.2 — Ek örnek: 30 kişilik sınıfa numara

Soru: 30 öğrencinin her birine farklı bir ikilik numara verilecek. En az kaç bit gerekir?
Verilenler: Ayırt edilecek nesne sayısı = 30. **İstenen:** En küçük n.
Yöntem ve neden: n bitle 2^n farklı desen var; her öğrenciye ayrı bir desen düşmesi için $2^n \geq 30$ olmalı. Bu eşitsizliği sağlayan en küçük n'yi arıyoruz.

Çözüm: $2^4 = 16 < 30$, yetmez. $2^5 = 32 \geq 30$, yeter.
Sonuç: 5 bit. (2 desen boşta kalır, sorun değil.)
En sık hata: $2^n = 30$'u tam sağlayan n aramak ya da n = 30/2 = 15 demek. 2^n'nin 30'a eşit olması gerekmez, **en az** 30 olması gerekir.
Başka nerede: Bölüm 6'da bellek adresi sayısı, Bölüm 4'te renk sayısı için birebir aynı mantığı kullanacaksın. Matematikçe yazımı: $n = \lceil \log_2 30 \rceil = 5$.

▶ ÖRNEK 1.3 — Yarı çözümlü: ASCII kaç karakter gösterir?

Bölüm 3'te göreceğin ASCII, her karakter için **7 bit** kullanır.

1. Formül: 7 bit → 2^7 farklı desen.
2. $2^7 = 2^6 \cdot 2 = 64 \cdot 2 = \ldots$
3. En küçük kod 0 ise en büyük kod ...

Son iki adımı sen tamamla. (Cevap bölüm sonunda.)

✎ KENDİN ÇÖZ

Bir oyunda 200 farklı eşya türü var ve her tür bir bit deseniyle saklanacak. (a) En az kaç bit gerekir? (b) O bit sayısıyla kaç tür daha eklenebilir? (c) Eşya türü 300'e çıkarsa kaç bit gerekir?

✎ DURUP DÜŞÜN

3. 8 bitle 256 değer var. 16 bit, 8 bitin iki katı; peki 16 bitle gösterilen değer sayısı da 256'nın iki katı mı?

Bağlantılar

- **2^n kuralı** Bölüm 4'te "b bit/piksel → 2^b renk", Bölüm 6'da "n adres biti → 2^n bayt" olarak birebir geri dönecek. Üçü de aynı formül, sadece "değer" yerine renk ya da adres yazıyoruz.
- **"Kural anlamı belirler"** fikri Bölüm 3'te kod sayfası karışıklığının (bozuk Türkçe karakterler) tam açıklaması olacak.
- Kodlama zincirindeki "sayısallaştırma" adımını Bölüm 5'te ses için (örnekleme + nicemleme) ayrıntılı göreceksin.

⚠ SINAVDA DİKKAT

- Veri = **ham** değer; bilgi = bir soruyu cevaplamak için **yorumlanmış** veri. Fark yaratan şey **bağlam**.
- 1 bayt = **8** bit (4 bit değil, 10 bit değil).
- n bit → **2^n** değer, en büyük işaretsiz değer **$2^n - 1$**. "8 bitle en büyük sayı?" sorusunun cevabı 255, "kaç değer?" sorusunun cevabı 256.
- Gerekli bit sayısı sorularında 2^n'nin **eşit veya büyük** olması yeter.

Karıştırılan	Fark
kaç farklı değer	2^n

en büyük değer	$2^n - 1$
kaç bit gerekir (N nesne)	$2^n \geq N$ olan en küçük n

❖ BU KONUDAN NASIL SORU GELİR?

1. Kavramsal: *"Explain the difference between data and information with an example."* İpucu: "difference" kelimesi → iki tanım + bir örnek + bağlamın rolü. Her iki tanımda da anahtar kelimeyi (raw / interpreted) kullan.
2. Hesap: *"How many different values can be represented with 12 bits?"* ya da *"What is the minimum number of bits to encode 100 symbols?"* İpucu: "how many values" → 2^n hesapla; "minimum number of bits" → $2^n \geq N$ olan en küçük n'yi bul.
3. Çoktan seçmeli tuzak: "8 bitlik işaretsiz sayının alabileceği en büyük değer" şıklarında 256 da 255 de olur. Doğru cevap 255.

✘ SIK YAPILAN HATA

Yanlış: "n bitle n^2 değer gösterilir." → **Doğrusu:** 2^n. → **Neden:** Her bit bağımsız 2 seçenek sunar; seçenekler çarpılır ($2 \cdot 2 \cdot \ldots$), bit sayısı ile çarpılmaz. 3 bitle $3^2 = 9$ değil, $2^3 = 8$ desen var; listeleyip say.
Yanlış: "16 bit, 8 bitin iki katı; o hâlde değer sayısı da iki katı (512)." → **Doğrusu:** $2^{16} = 65{,}536$, yani 256 kat. → **Neden:** Her ek bit katlar; 8 ek bit $2^8 = 256$ kat demek.
Yanlış: "Sayı gördüğüm her şey veridir." → **Doğrusu:** "Sınıf ortalaması 78" bir sayı içerir ama bilgidir. → **Neden:** Ölçüt biçim değil, yorumlanmış olması.

⚡ HIZLI TEKRAR

1. Veri: ham değerler. Bilgi: soruya cevap verecek şekilde yorumlanmış veri. Bağlam veriyi bilgiye çevirir.
2. Beş veri türü: sayı, metin, görüntü, ses, video. Hepsi bitlere kodlanır.
3. Fiziksel olay → algılama → sayısallaştırma → bit dizisi; geri yön kod çözme.
4. Bit 0/1; 8 bit = 1 bayt; bitlerin anlamını kodlama kuralı belirler.
5. n bit → 2^n değer ($0 \ldots 2^n - 1$); her ek bit değer sayısını ikiye katlar.

Ezberle: $2^0 \ldots 2^{10}$ ve $2^{16} = 65{,}536$. **Türetebil:** 2^n kuralının nedeni.
Bu bölümden sonra şunları yapabiliyor olmalısın: veri/bilgi örneklerini gerekçesiyle sınıflandırmak; N nesne için gereken bit sayısını bulmak; aynı bit deseninin farklı anlamlara gelebileceğini bir örnekle açıklamak.

Bölüm 1 cevapları

Durup düşün 1. İstasyonun ham kayıtları **veri**dir. Bu kayıtlardan "bugünün en yüksek sıcaklığı 23 °C" çıkarıldığında bilgi olur.

Durup düşün 2. Evet. "Assignment grade: 85" öğrenci için bilgidir (kaç aldığını cevaplar), hocanın ortalama hesabında ise 30 notan biri olan bir veri noktasıdır. Yani "veri mi bilgi mi" sorusu bakan kişiye ve sorduğu soruya bağlıdır.

Durup düşün 3. Hayır. $2^{16} = 65{,}536 = 256 \cdot 256$, yani 256'nın iki katı değil **256 katı**. Bit sayısını iki katına çıkarmak, değer sayısının **karesini** almak demektir: $(2^8)^2 = 2^{16}$.

Yarı çözümlü örnek. $2^7 = 128$ farklı kod; en küçük kod 0 olduğuna göre en büyük kod 127. (Slayt D1 s. 32 de "ASCII 7 bit, 128 kod" diyor.)

Kendin çöz. (a) $2^7 = 128 < 200$, $2^8 = 256 \geq 200 \rightarrow$ **8 bit.** (b) $256 - 200 =$ **56** tür daha eklenebilir. (c) $2^8 = 256 < 300$, $2^9 = 512 \geq 300 \rightarrow$ **9 bit.**

BÖLÜM 2

Sayı Sistemleri ve Dönüşümler

◆ **BU BÖLÜMDE** Okuma süresi: 70 dk Zorluk: ●●○

- Konumsal sayı sisteminin mantığını ve genel formülü $(d_k \ldots d_1 d_0)_b = \Sigma\, d_i \cdot b^i$
- Bilgisayarların neden ikilik sistemi seçtiğini (donanım ve gürültü payı)
- Onaltılık sistemi, A–F rakamlarını, 0x önekini ve "4 bit = 1 onaltılık rakam" kuralını
- İkilik, onluk ve onaltılık arasında altı yönde dönüşüm yapmayı ve sonucu kontrol etmeyi
- Slayt 22, 24–29'daki bütün alıştırmaları adım adım

SINAV ÖNCELİĞİ: YÜKSEK ❖ Kaynak: D1 s. 11–29

Sınav önceliği yüksek: öğrenme çıktılarında açıkça "Convert among binary, decimal, and hexadecimal" yazıyor (D1 s. 3), bölüm 19 slayt sürüyor ve s. 26–29 doğrudan alıştırma. Bu konudan neredeyse kesin işlem sorusu bekle.

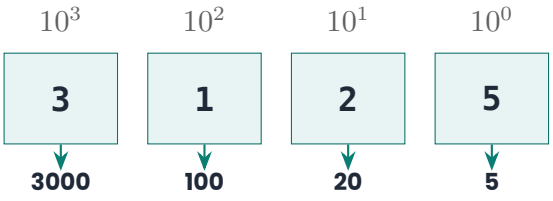
→ BU KONUYA NEDEN GEÇİYORUZ?

Bölüm 1'de her şeyin bitlere döndüğünü ve n bitin 2^n desen verdiğini gördük. Ama elimizde hâlâ bir boşluk var: **01111101** desenini görünce bunun "125" olduğunu nasıl söylüyoruz? Ya da bir hata ekranındaki **0x00000050** ne demek? Bu bölüm, bit desenlerini sayı olarak okumanın kuralını (konumsal sistem) ve uzun bit dizilerini insanların okuyabileceği hâle getiren kısaltmayı (onaltılık sistem) veriyor.

2.1 3125'teki "3" neden 3000 eder?

❖ Kaynak: D1 s. 12–13

Romalılar 3125'i **MMMCXXV** diye yazardı. Burada her sembolün değeri sabit: M her yerde 1000, X her yerde 10. Bu sistemle çarpma yapmayı dene; kâbus. Bizim kullandığımız sistemin süper gücü başka: **aynı rakam, durduğu yere göre farklı değer taşır.**



$$3125 = 3 \cdot 1000 + 1 \cdot 100 + 2 \cdot 10 + 5 \cdot 1$$

Şekil 2.1: D1 s. 13: Her basamağın değeri = rakam × basamak ağırlığı. Ağırlıklar sağdan sola 10'un kuvvetleri olarak artar.

∑ TANIM / FORMÜL — Konumsal sayı sistemi (positional number system)

- Bir rakamın değeri, **rakamın kendisine ve konumuna** bağlıdır. *"A digit's value depends on the digit and its position."*
- **Taban (base)**, her konumda hangi rakamların kullanılabileceğini belirler.
- **Basamak ağırlıkları (place weights)** sağdan sola tabanın kuvvetleri olarak artar: b^0, b^1, b^2, ...
- Tabanı belirtmek için **alt indis** (1010_2) ya da **önek** (0x2A) kullanılır; aksi hâlde "10" on mu, iki mi belli olmaz.

Son madde küçük gibi görünür ama sınavda can kurtarır: "10" yazısı onlukta on, ikilikte iki, onaltılıkta on altıdır. Tabanı yazmazsan cevabın belirsiz kalır.

2.2 Bütün tabanlar için tek formül

❖ Kaynak: D1 s. 14

3125 örneğini genelleştirelim. Rakamları sağdan numaralayalım: en sağdaki d_0, bir solundaki d_1, ... en soldaki d_k. Taban b olsun. O zaman:

∑ TANIM / FORMÜL — Taban gösterimi (base notation)

$(d_k \ldots d_2\ d_1\ d_0)_b = d_k \cdot b^k + \ldots + d_2 \cdot b^2 + d_1 \cdot b^1 + d_0 \cdot b^0 = \Sigma\ d_i \cdot b^i$ (i = 0'dan k'ya)

- d_i: i. konumdaki rakam (sağdan, **0'dan** başlayarak sayılır)
- b: taban; b^i: o konumun ağırlığı
- **Koşul:** her rakam tabandan küçük olmalı: $0 \le d_i \le b - 1$. *"The digit value in every position must be smaller than the base."*

Taban 10	Taban 2	Taban 16
0–9	0–1	0–9, A–F

Neden rakam tabandan küçük olmalı? Onlukta "10" diye bir rakam yok; 9'dan sonra bir üst basamağa elde verip "10" yazıyoruz. Eğer bir basamağa b veya daha büyük bir rakam koyabilseydin, aynı sayının iki farklı yazılışı olurdu (onlukta "1|10" ile "20" aynı değer olurdu). Kural, her sayının **tek** bir yazılışı olmasını garanti eder. Bu yüzden 1012_2 ya da 19_8 geçersiz yazılardır.

✎ DURUP DÜŞÜN

1. Formülde konumları neden 0'dan başlatıyoruz da 1'den başlatmıyoruz? (İpucu: en sağdaki basamağın ağırlığı ne olmalı?)

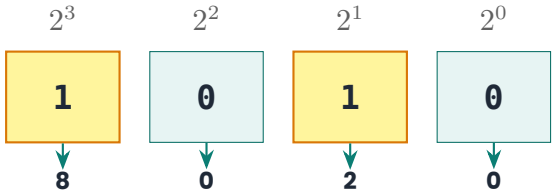
✗ SIK YAPILAN HATA — Slayttaki küçük gösterim tutarsızlığı

D1 s. 14'te formülün sol tarafında alt indis **β** (beta), sağ tarafında taban **b** olarak yazılmış ve toplamın sınırları (i = 0...k) verilmemiş. İkisi aynı şeyi, yani tabanı kastediyor. Sınavda **b** yazman yeterli; toplamın sınırını da eklersen kusursuz olur.

2.3 İkilik sistem ve donanımın sevgisi

❖ Kaynak: D1 s. 15–16

Taban 2'de yalnızca iki rakam var: 0 ve 1. Ağırlıklar 2'nin kuvvetleri: 1, 2, 4, 8, 16...



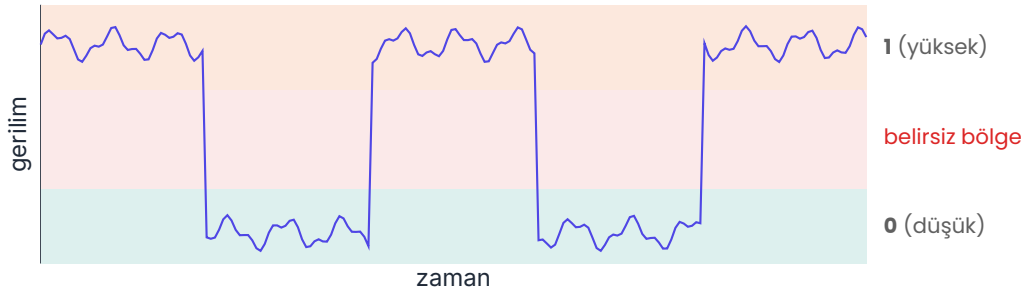
$$1010_2 = 8 + 0 + 2 + 0 = 10_{10}$$

Şekil 2.2: D1 s. 15: İkilikte rakam ya 0 ya 1 olduğu için "rakam × ağırlık" çarpımı ya ağırlığın kendisi ya 0'dır. Sadece 1 olan (sarı) basamakların ağırlıklarını topla.

Peki bilgisayar neden bizim gibi onluk sistem kullanmıyor? Çünkü donanım iki durumu **güvenilir biçimde** ayırt edebilir (*"Hardware can distinguish two physical states reliably."*):

1. Gerilim (voltaj) düşük ya da yüksek olabilir.
2. Bir transistör kapalı ya da açık olabilir.
3. Manyetik yön ya da ışık seviyesi iki aralığa eşlenebilir.
4. Gürültü payları (**noise margins**) iki durumu güvenle ayırmaya yardım eder.

Dördüncü madde işin özü. Gerçek bir telde gerilim hiçbir zaman tam 0 V ya da tam "yüksek" değildir; ısı, komşu teller ve parazit yüzünden titrer. İki seviye kullanırsan aralarında geniş bir **güvenlik boşluğu** bırakabilirsin:



Şekil 2.3: Gürültülü bir sinyal bile iki geniş aralıktan birinde kaldığı sürece doğru okunur: 1, 0, 1, 0, 1. Aradaki "belirsiz bölge" güvenlik payıdır. (Temsili çizim; gerçek gerilim değerleri teknolojiye göre değişir.)

Aynı teli 10 seviyeye (0–9) bölseydin her aralık daralır, küçük bir parazit "6"yı "7"ye çevirebilirdi. İkilik sistem bu yüzden donanım için en sağlam seçimdir.

★ MERAKLISINA (SINAV DIŞI)

1950'lerde Sovyetler Birliği'nde **Setun** adında, üç durumlu (−1, 0, +1) "üçlük" mantıkla çalışan bir bilgisayar gerçekten üretildi. Matematiksel olarak bazı avantajları vardı ama üç seviyeyi güvenilir ayırt eden ucuz donanım yapmak zordu; dünya ikilikte kaldı.

2.4 Tabanlar tablosu: "on" sayısını dört dilde yazmak

❖ Kaynak: D1 s. 17

Sistem	Taban	Rakamlar	"On" sayısı
İkilik (**binary**)	2	0, 1	1010_2
Sekizlik (**octal**)	8	0–7	12_8
Onluk (**decimal**)	10	0–9	10_{10}
Onaltılık (**hexadecimal**)	16	0–9, A–F	A_{16}

Kontrol edelim: $12_8 = 1 \cdot 8 + 2 = 10$ ✓; $A_{16} = 10$ ✓. Aynı miktar, dört farklı yazı.

2.5 Onaltılık sistem: bitlerin kısaltması

❖ Kaynak: D1 s. 18–21, 23

2.5.1 10'dan 15'e kadar rakamlar nereden geliyor?

Taban 16'da her basamakta 0'dan 15'e kadar 16 farklı rakam olmalı. Ama elimizde tek karakterlik rakam olarak yalnızca 0–9 var. Çözüm: harfleri ödünç almak.

∑ TANIM / FORMÜL — Onaltılık rakamlar

A = 10, B = 11, C = 12, D = 13, E = 14, F = 15 (D1 s. 19)
Ağırlıklar 16'nın kuvvetleri: $16^0 = 1$, $16^1 = 16$, $16^2 = 256$, $16^3 = 4096$.

▶ ÖRNEK 2.1 — Slayt örneği [D1 s. 18]: $3AB_{16}$ → onluk

Yöntem: Formül: $\Sigma\, d_i \cdot 16^i$. Önce harfleri sayıya çevir: A = 10, B = 11.

Konum	2	1	0
Rakam	3	A (10)	B (11)
Ağırlık	$16^2 = 256$	$16^1 = 16$	$16^0 = 1$
Katkı	768	160	11

$3AB_{16} = 768 + 160 + 11 = \mathbf{939_{10}}$
En sık hata: A'yı 1, B'yi 2 sanmak ya da harfi olduğu gibi bırakıp hesaplayamamak.

▶ ÖRNEK 2.2 — Slayt örnekleri [D1 s. 19]: 0x2A, 0x7D, 0xFF

- 0x2A = $2 \cdot 16 + 10 = 32 + 10 =$ **42**
- 0x7D = $7 \cdot 16 + 13 = 112 + 13 =$ **125**
- 0xFF = $15 \cdot 16 + 15 = 240 + 15 =$ **255**

0xFF = 255'i özellikle aklında tut: 8 bitin (1 baytın) alabileceği en büyük değer. Bölüm 4'te RGB'de, Bölüm 6'da adreslerde tekrar göreceksin.

2.5.2 Neden tam olarak 4 bit = 1 onaltılık rakam?

❖ Kaynak: D1 s. 20

Bölüm 1'deki kural: 4 bit → $2^4 =$ **16** desen. Onaltılık sistemde de tam **16** rakam var. Desenler ile rakamlar arasında birebir eşleme kurabiliriz:

İkilik	Onluk	Hex	İkilik	Onluk	Hex
0000	0	0	1000	8	8
0001	1	1	1001	9	9
0010	2	2	1010	10	A
0011	3	3	1011	11	B
0100	4	4	1100	12	C
0101	5	5	1101	13	D
0110	6	6	1110	14	E
0111	7	7	1111	15	F

Tablo 2.4: Bu tablo, haftanın en çok işine yarayacak tablosu. Sınavda kâğıdın köşesine ilk iş olarak bunu yaz; 8-4-2-1 ağırlıklarını kullanarak saniyeler içinde üretebilirsin.

Bu birebir eşleme sayesinde ikilik ↔ onaltılık dönüşümü **hiç hesap gerektirmez**, sadece tablo bakmaktır. Slaytın alt notu: *"Every group of four bits can be written as one hexadecimal digit."*

Onaltılığın asıl işi bu: uzun bit dizilerini **4 kat kısaltmak** (*"Hexadecimal makes long bit sequences shorter and easier to read."*). 32 bitlik bir adresi 32 tane 0–1 yerine 8 karakterle yazarsın. D1 s. 21'deki kullanım alanları:

1. Bellek adresleri ve hata kodları
2. Makine kodu ve hata ayıklama (**debugging**) çıktıları
3. RGB renk değerleri (#RRGGBB, Bölüm 4)
4. Ağ protokollerindeki alanlar

2.5.3 0x öneki

❖ Kaynak: D1 s. 23

Programlama dilleri ve hata ayıklama araçları onaltılık değerleri **0x** önekiyle işaretler: 0xFF = 255. Önek tabanı açıkça belirtir. D1 s. 23'teki Windows "mavi ekran" görüntüsündeki **STOP: 0x00000050** satırı 8 onaltılık rakamlık yani 32 bitlik bir hata kodudur; 0x50 = 5 · 16 = 80.

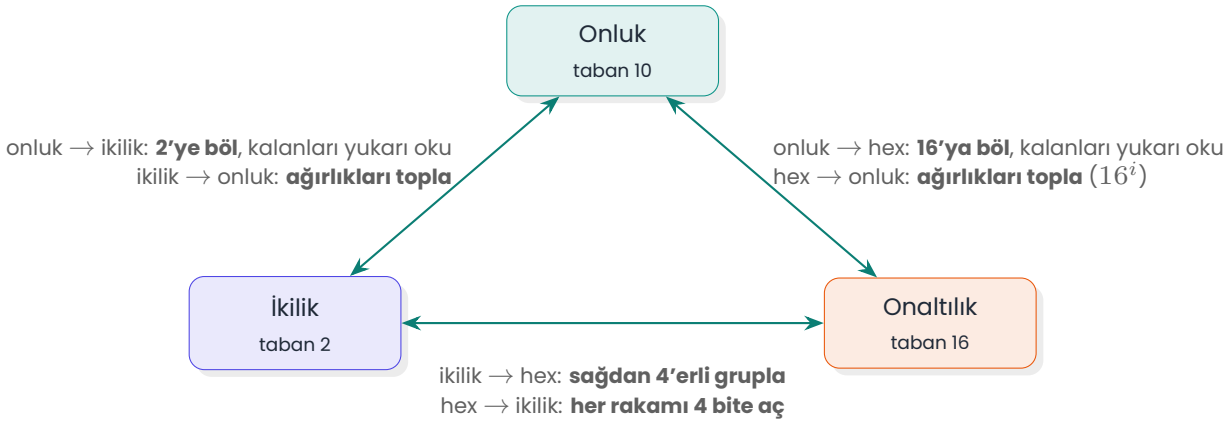
✎ **DURUP DÜŞÜN**

2. 0x00000050 ile 0x50 aynı sayı mı? Baştaki sıfırlar neden yazılmış olabilir?

2.6 Dönüşümler: altı yön, üç yöntem

❖ Kaynak: D1 s. 22, 24–27

D1 s. 27 bütün stratejileri tek tabloda topluyor. Önce resmin bütününe bakalım:



Şekil 2.4: D1 s. 27'deki dönüşüm stratejileri (onaltılık → onluk yönü slaytın s. 18'deki örneğinden). Onluğa giden ok her zaman "topla", onluktan çıkan ok her zaman "böl"; ikilik ile onaltılık arasında ise sadece gruplama var.

2.6.1 İkilik → onluk: ağırlıkları topla

İkilik sayının altına sağdan başlayarak 1, 2, 4, 8, ... yaz; **1 olan** basamakların ağırlıklarını topla.

▶ ÖRNEK 2.3 — Slayt alıştırması [D1 s. 26]: Beş ikilik sayı

İkilik	Ağırlık tablosu (1 olanlar)	Toplam
101_2	4 + 1	**5**
1011_2	8 + 2 + 1	**11**
10100_2	16 + 4	**20**
111010_2	32 + 16 + 8 + 2	**58**
1010001_2	64 + 16 + 1	**81**

Son satırı açık yazalım:

Konum	6	5	4	3	2	1	0
Bit	1	0	1	0	0	0	1
Ağırlık	64	32	16	8	4	2	1
Katkı	64	0	16	0	0	0	1

Kontrol (slaytın ipucu): *"The weight of the leftmost 1 bit gives the magnitude range of the result."* 1010001'de en soldaki 1'in ağırlığı 64, o hâlde sonuç 64 ile 127 arasında olmalı (bir sonraki ağırlık 128'den küçük). 81 bu aralıkta ✓.

2.6.2 Onluk → ikilik: 2'ye böl, kalanları yukarı doğru oku

❖ Kaynak: D1 s. 24

Yöntemi önce uygulayalım, sonra **neden** işe yaradığını görelim.

▶ ÖRNEK 2.4 — Slayt örneği [D1 s. 24]: 125_{10} → ikilik



Bölüm 0 olunca dur. Kalanları **aşağıdan yukarıya** okuyunca: **$125_{10} = 1111101_2$**
Kontrol: 64 + 32 + 16 + 8 + 4 + 1 = 125 ✓

Neden yukarı doğru okuyoruz? Bir sayıyı 2'ye böldüğünde kalan, sayının **çift mi tek mi** olduğunu söyler. İkilik bir sayıda tekliği sadece en sağdaki bit belirler (diğer bütün ağırlıklar 2, 4, 8... çifttir). Yani ilk kalan d_0'dır.

Daha matematikçe: 125 = 2 · 62 + **1**. Bölüm 62, sayının "en sağdaki biti kesilmiş hâli"dir. Aynı işlemi 62'ye uygulayınca d_1'i buluruz: 62 = 2 · 31 + **0**. Her bölme, sayının sağından bir bit "koparır". İlk kopan bit en sağdakidir; bu yüzden yazarken listeyi tersten okuruz.

+ EK BİLGİ (KAYNAK DIŞI) — İkinci yöntem: en büyük kuvveti çıkar

Bu yöntem slaytlarda yok ama küçük sayılarda daha hızlı. 125 için: 125'e sığan en büyük 2 kuvveti 64 → kalan 61; 32 → 29; 16 → 13; 8 → 5; 4 → 1; 2 sığmaz (0); 1 → 0. Kullanılanlara 1, kullanılmayanlara 0: 64·32·16·8·4·2·1 → 1111101 ✓. Sınavda slayttaki bölme yöntemini göstermen istenirse onu kullan; bu yöntem kendi kontrolün için.

2.6.3 Onluk → onaltılık: 16'ya böl

❖ Kaynak: D1 s. 25

Tamamen aynı mantık, bu sefer 16'ya bölüyoruz. Tek fark: kalan 0 ile 15 arasında olabilir ve 10–15 arası kalanları **harfe** çevirmeliyiz.

▶ ÖRNEK 2.5 — Slayt örneği [D1 s. 25]: 125_{10} → onaltılık

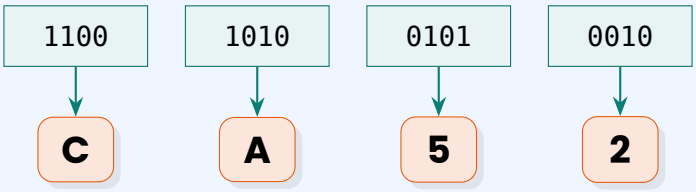
1. 125 ÷ 16 = 7, kalan 13 (çünkü 7 · 16 = 112, 125 − 112 = 13)
2. 7 ÷ 16 = 0, kalan 7
3. 13 onaltılıkta **D**

Yukarı doğru oku: **$125_{10} = 7D_{16}$**
Kontrol: 7 · 16 + 13 = 125 ✓. Ayrıca s. 24'teki sonuçla tutarlı mı? 1111101_2 → sağdan 4'erli: 0111 1101 → 7 D ✓. İki farklı yoldan aynı cevap.
En sık hata: Kalanı "13" diye iki rakamla yazıp **713** sonucu vermek. Her kalan **tek bir** onaltılık rakam olmalı.

2.6.4 İkilik → onaltılık: sağdan 4'erli grupla

❖ Kaynak: D1 s. 22

▶ ÖRNEK 2.6 — Slayt örneği [D1 s. 22]: 1100101001010010_2 → onaltılık



$$1100101001010010_2 = CA52_{16}$$

16 bit, tam 4 gruba bölündü; her grubu tablodan oku: 1100 = 12 = C, 1010 = 10 = A, 0101 = 5, 0010 = 2.

Neden sağdan gruplanır? Çünkü ağırlıklar sağdan başlar. En sağdaki 4 bit 2^0…2^3 ağırlıklarını, sonraki 4 bit 2^4…2^7'yi (yani 16^1 katını) taşır. Soldan gruplarsan basamaklar kayar. Bit sayısı 4'ün katı değilse **en soldaki gruba başına sıfır eklersin**; baştaki sıfırlar değeri değiştirmez.

✗ SIK YAPILAN HATA — Soldan gruplama tuzağı

Yanlış: 1111101_2'yi soldan grupla: 1111 | 101 → F5. → **Doğrusu:** sağdan grupla: 111 | 1101 → **0**111 | 1101 → **7D**. → **Neden yanlış:** F5 = 245, oysa sayı 125. Soldan gruplayınca son grup eksik kalıyor ve bütün basamaklar bir konum kayıyor.

2.6.5 Onaltılık → ikilik: her rakamı 4 bite aç

Ters yönde her rakamı tablodan 4 bitlik karşılığıyla değiştirirsin. **Her rakam tam 4 bit** olmalı; 2 → "10" değil **0010**.

▶ ÖRNEK 2.7 — Ek örnek: 0x3E7 → ikilik → onluk

Adım 1 (hex → ikilik): 3 → 0011, E (14) → 1110, 7 → 0111. Sonuç: $0011\ 1110\ 0111_2 = 1111100111_2$ (baştaki sıfırlar atılabilir).
Adım 2 (hex → onluk, kontrol için): $3 \cdot 256 + 14 \cdot 16 + 7 = 768 + 224 + 7 =$ **999**.
Adım 3 (ikilik → onluk, çapraz kontrol): $512 + 256 + 128 + 64 + 32 + 4 + 2 + 1 = 999$ ✓.
En sık hata: Ortadaki rakamı 3 bitle yazıp (ör. 3 → 11) sonucu kaydırmak.

▶ ÖRNEK 2.8 — Ek örnek: 200_{10} → ikilik ve onaltılık

Neden bu yolu seçiyoruz: Önce ikiliği bölme yöntemiyle bulup, onaltılığı gruplamayla almak; 16'ya bölmeye göre ayrıca kontrol imkânı verir.

Bölme	Bölüm	Kalan
200 ÷ 2	100	0
100 ÷ 2	50	0
50 ÷ 2	25	0
25 ÷ 2	12	1
12 ÷ 2	6	0
6 ÷ 2	3	0
3 ÷ 2	1	1

1 ÷ 2 0 1

Yukarı oku: **11001000_2**. Grupla: 1100 | 1000 → **$C8_{16}$**.
Kontrol: 200 ÷ 16 = 12 kalan 8 → C8 ✓; 128 + 64 + 8 = 200 ✓.

▶ ÖRNEK 2.9 — Yarı çözümlü: 1011011_2 → onaltılık ve onluk

1. Bit sayısı 7; 4'ün katı değil. Sağdan grupla ve başa 0 ekle: 0101 | 1011
2. Gruplar: 0101 = ..., 1011 = ...
3. Onaltılık sonuç: $...{}_{16}$
4. Onluk: 5 · 16 + ... = ...

Boşlukları sen doldur.

2.7 Slayt 28–29: on dönüşüm, tam çözüm

❖ Kaynak: D1 s. 28–29

D1 s. 28 soruları, s. 29 yalnızca sonuçları veriyor. Ara adımlar aşağıda.

Onluk → ikilik (2'ye böl, yukarı oku):

No	Sayı	Kalanlar (ilk bölmeden son bölmeye)	Yukarıdan okunan sonuç	Kontrol
01	11	1, 1, 0, 1	**1011**	8+2+1
02	27	1, 1, 0, 1, 1	**11011**	16+8+2+1
03	54	0, 1, 1, 0, 1, 1	**110110**	32+16+4+2
04	113	1, 0, 0, 0, 1, 1, 1	**1110001**	64+32+16+1
05	273	1, 0, 0, 0, 1, 0, 0, 0, 1	**100010001**	256+16+1

Örnek olarak 273'ün bölmeleri: 273→136 (k 1), 136→68 (0), 68→34 (0), 34→17 (0), 17→8 (1), 8→4 (0), 4→2 (0), 2→1 (0), 1→0 (1). Kalanları **ters** sırada yaz: 100010001.

İkilik → onluk (ağırlıkları topla):

No	İkilik	1 olan basamakların ağırlıkları	Sonuç
06	101	4 + 1	**5**
07	1011	8 + 2 + 1	**11**
08	10100	16 + 4	**20**
09	111010	32 + 16 + 8 + 2	**58**
10	1010001	64 + 16 + 1	**81**

Dikkat ettin mi? 01 ile 07 birbirinin tersi: 11 = 1011_2 ve 1011_2 = 11. Kendi cevabını her zaman ters yönde dönüştürerek kontrol edebilirsin.

✎ KENDİN ÇÖZ

(a) 156_{10}'u ikiliğe ve onaltılığa çevir. (b) 0x2F9'u onluğa ve ikiliğe çevir. (c) 10011_2 + 0x0D toplamını onlukta bul. (Bütün adımları göster.)

+ EK BİLGİ (KAYNAK DIŞI) — Sekizlik ↔ ikilik: 3'erli grupla

Slayt s. 17'de sekizlik sistem tabloda var ama dönüşümü yok. Aynı fikir: $2^3 = 8$ olduğundan **3 bit = 1 sekizlik rakam**. 1111101_2 → sağdan 3'erli: 1 | 111 | 101 → 001 | 111 | 101 → **175_8**. Kontrol: $1 \cdot 64 + 7 \cdot 8 + 5 = 125$ ✓.

Bağlantılar

- **$2^4 = 16$** (Bölüm 1) olmasaydı "4 bit = 1 hex rakam" kısayolu olmazdı.
- Bölüm 3'te ASCII tablosu hem onluk hem onaltılık sütunlarla verilir (A = 65 = 0x41), Unicode kod noktaları ise U+0041 gibi onaltılık yazılır.
- Bölüm 4'te #RRGGBB renk kodları 3 tane 2 haneli onaltılık sayıdır (FF = 255).
- Bölüm 6'da bellek adresleri hep onaltılık ve 0x önekiyle yazılır; her hex rakam 4 adres biti demektir.

⚠ SINAVDA DİKKAT

- Tabanı mutlaka yaz: 1010_2, 12_8, A_{16}, 0x2A.
- Rakam < taban: 2 ikilikte, 8 sekizlikte, G onaltılıkta **geçersiz**.
- Konumlar **sağdan ve 0'dan** sayılır; en sağdaki ağırlık $b^0 = 1$.
- Bölme yönteminde kalanlar **aşağıdan yukarı** okunur; ilk kalan en sağdaki rakamdır.
- İkilik → hex: **sağdan** 4'erli grupla, eksik grubun **başına** 0 ekle. Hex → ikilik: her rakam **tam 4 bit**.
- 10–15 arası kalan **harf** olur (D, E, F...), iki haneli yazılmaz.

Dönüşüm	Yöntem
ikilik → onluk	ağırlıkları topla
onluk → ikilik	2'ye böl, kalanları yukarı oku
ikilik → hex	sağdan 4'lü grupla
hex → ikilik	her rakam → 4 bit
onluk → hex	16'ya böl, kalanları yukarı oku
hex → onluk	$\Sigma\, d_i \cdot 16^i$

❖ BU KONUDAN NASIL SORU GELİR?

1. İşlem (kesin gibi): *"Convert 173 to binary and hexadecimal. Show your work."* İpucu: "show your work" → bölme tablosunu tam yaz; puanın çoğu ara adımlarda. Sonra kontrol satırı ekle.
2. Kavramsal: *"Why do computers use binary? Why is hexadecimal used?"* İpucu: iki soruyu ayrı cevapla: ikilik → donanım iki durumu güvenilir ayırır (voltaj, transistör, gürültü payı); onaltılık → 4 bit = 1 rakam, uzun bit dizilerini kısaltır (adresler, renkler).
3. Hata bulma: "Bir öğrenci $1111101_2 = F5_{16}$ yazmış. Hatayı bul." İpucu: gruplama yönü.

4. Geçerlilik: "Aşağıdakilerden hangisi geçerli bir sayı değildir? 1021_2, 777_8, $1A_{16}$, 99_{10}" İpucu: rakam < taban kuralı → 1021_2.

✘ SIK YAPILAN HATA

Yanlış: Kalanları yukarıdan aşağı okumak (125 → 1011111). → **Doğrusu:** aşağıdan yukarı, 1111101. → **Neden:** İlk kalan en sağdaki bittir (sayının tek/çift olduğunu söyler).
Yanlış: Konumları 1'den numaralamak ve en sağdaki bitin ağırlığını 2 almak (101_2 = 8 + 2 = 10). → **Doğrusu:** 4 + 1 = 5. → **Neden:** En sağdaki ağırlık $2^0 = 1$.
Yanlış: Hex → onluk çevirirken 16'nın kuvvetleri yerine 10'un kuvvetlerini kullanmak (3AB → 3 · 100 + …). → **Doğrusu:** 3 · 256 + 10 · 16 + 11. → **Neden:** Ağırlıklar tabanın kuvvetleridir.
Yanlış: Onaltılık rakamı 3 bitle açmak (0x52 → 101 10). → **Doğrusu:** 0101 0010. → **Neden:** 16 rakamı ayırt etmek için 4 bit gerekir; eksik bit diğer grupları kaydırır.

⚡ HIZLI TEKRAR

1. Konumsal sistem: değer = Σ rakam × taban^konum; konumlar sağdan 0'dan.
2. Rakamlar 0…b − 1; onaltılıkta A–F = 10–15.
3. İkilik: donanım iki durumu gürültüye rağmen güvenle ayırt eder.
4. Onaltılık: $2^4 = 16$ → 4 bit = 1 rakam; uzun bit dizilerini kısaltır; 0x öneki.
5. Onluğa = topla; onluktan = böl ve yukarı oku; ikilik ↔ hex = sağdan 4'lü grupla.
6. Her dönüşümü ters yönden kontrol et.

Ezberle: 0–15 için ikilik/hex tablosu (8-4-2-1'den türetilir), $16^2 = 256$, 0xFF = 255. **Türetebil:** yukarı okumanın nedeni, sağdan gruplamanın nedeni.
Bu bölümden sonra şunları yapabiliyor olmalısın: altı yönde dönüşüm yapıp kontrol etmek; ikilik ve onaltılığın neden kullanıldığını açıklamak; geçersiz bir sayı yazımını fark etmek.

Bölüm 2 cevapları

Durup düşün 1. En sağdaki basamağın ağırlığı 1 olmalı (birler basamağı). $b^0 = 1$ olduğu için konumu 0'dan başlatınca formül kendiliğinden doğru çıkar. 1'den başlatsaydık en sağ ağırlık $b^1 = b$ olurdu ve her şey bir kat büyük çıkardı.

Durup düşün 2. Evet, aynı sayı (80). Baştaki sıfırlar değeri değiştirmez. Hata kodu 32 bitlik bir alanda saklandığı için alanın tamamı, yani 8 onaltılık rakam gösterilmiş; böylece bütün kodlar aynı genişlikte ve okunması kolay oluyor.

Yarı çözümlü örnek. 0101 = 5, 1011 = 11 = B → **$5B_{16}$**. Onluk: 5 · 16 + 11 = **91**. (Kontrol: 64 + 16 + 8 + 2 + 1 = 91 ✓)

Kendin çöz.

(a) 156 ÷ 2: kalanlar sırasıyla 0, 0, 1, 1, 1, 0, 0, 1 (bölümler 78, 39, 19, 9, 4, 2, 1, 0). Yukarı oku: **10011100_2**. Grupla: 1001 | 1100 → **$9C_{16}$**. Kontrol: 9 · 16 + 12 = 156 ✓; 128 + 16 + 8 + 4 = 156 ✓.

(b) 0x2F9 = 2 · 256 + 15 · 16 + 9 = 512 + 240 + 9 = **761**. İkilik: 2 → 0010, F → 1111, 9 → 1001 → 0010 1111 1001 = **1011111001_2**. Kontrol: 512 + 128 + 64 + 32 + 16 + 8 + 1 = 761 ✓.

(c) 10011_2 = 16 + 2 + 1 = 19; 0x0D = 13. Toplam **32**. Farklı tabanlardaki sayıları toplamadan önce aynı tabana çevirmek gerekir.

BÖLÜM 3

Metnin Temsili: ASCII'den UTF-8'e

◆ **BU BÖLÜMDE** Okuma süresi: 55 dk Zorluk: ●●○

- Bir dosyanın harfleri değil, harflerin **sayısal kodlarını** sakladığını
- ASCII'nin 7 bitlik yapısını, A = 65, a = 97, '0' = 48 üçlüsünü ve ASCII tablosunu okumayı
- 8 bitlik kod sayfalarının neden bozuk Türkçe karakterlere yol açtığını
- EBCDIC'in tarihsel rolünü
- Unicode **kod noktası** ile UTF-8/16/32 **kodlaması** arasındaki farkı ve karakter başına bayt hesabını

SINAV ÖNCELİĞİ: YÜKSEK ❖ Kaynak: D1 s. 30–41 / slayt 32–40

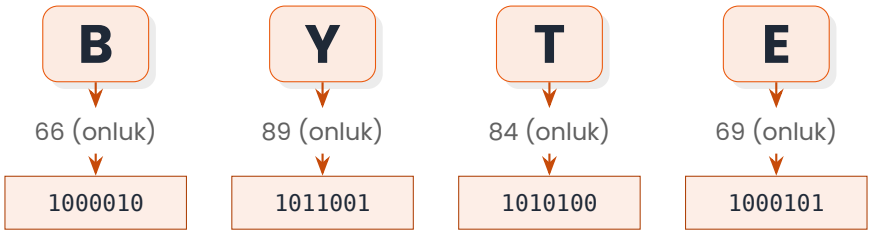
Sınav önceliği yüksek: öğrenme çıktısında "Explain the roles of ASCII, Unicode, and UTF encodings" var ve haftanın özet slaytı (D1 s. 66) "Unicode karakterin kimliğini, UTF baytlarını tanımlar" ayrımını vurguluyor. Hem tanım hem "kaç bayt?" hesabı gelebilir.

→ BU KONUYA NEDEN GEÇİYORUZ?

Artık bir bit desenini sayı olarak okuyabiliyoruz (Bölüm 2). Ama mesajlaşırken bilgisayar sayılarla değil harflerle, emojilerle uğraşıyor gibi görünüyor. İşin sırrı basit: **her karaktere bir numara verilir**, bilgisayar bu numarayı ikilik sayı olarak saklar. Bölüm 1'deki "kural anlamı belirler" fikri burada can alıcı hâle geliyor: Hangi numara hangi harf? Bu kurala **karakter kodlaması** diyoruz ve tarihte birkaç tane rakip kural olduğu için bugün hâlâ "Ã§" gibi bozuk harfler görüyoruz.

3.1 Bir dosya "BYTE" kelimesini nasıl saklar?

❖ Kaynak: D1 s. 31 / slayt 32



Şekil 3.1: D1 s. 31: Her harfin ayrı bir sayısal kodu var. Dosyaya çizilen harf şekli değil, bu 7 bitlik kodlar (bayt olarak) yazılır.

Slaytın üç mesajı:

- Her karakterin ayrı bir sayısal kodu vardır.
- ASCII'de B, Y, T ve E'nin farklı 7 bitlik değerleri vardır.
- Dosya bu kodların **baytlarını** saklar, görsel karakterleri değil (*"not the visual characters"*).

Kontrol edelim: $1000010_2 = 64 + 2 = 66$ ✓; $1011001_2 = 64 + 16 + 8 + 1 = 89$ ✓. Bölüm 2 burada işe yaradı.

3.2 ASCII: 128 karakterlik İngilizce alfabe

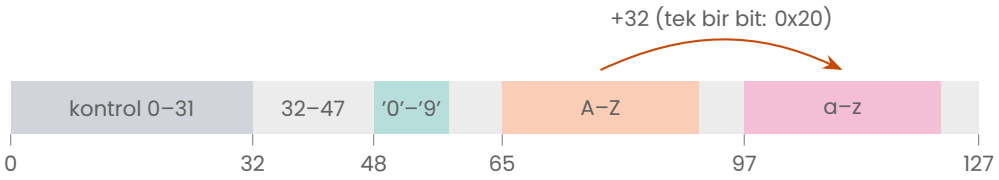
❖ Kaynak: D1 s. 32–33 / slayt 33–34

∑ TANIM / FORMÜL — ASCII (American Standard Code for Information Interchange)

- **7 bit** kullanır ve **128** kod tanımlar ($2^7 = 128$, kodlar 0–127). *"ASCII uses 7 bits and defines 128 codes."*
- İngilizce harfler, rakamlar, noktalama işaretleri ve **kontrol karakterleri** içerir.
- Büyük **A = 65**, küçük **a = 97**.
- UTF-8, ASCII kodlarını aynı tek baytla korur.

ASCII tablosunun tamamını ezberlemen gerekmez; birkaç "çapa" noktası ve tablonun düzeni yeter:

Aralık	İçerik	Örnek
0–31 ve 127	Kontrol karakterleri (görünmez)	10 = satır sonu (Line feed), 9 = Tab
32	Boşluk (space)	0x20
48–57	Rakam **karakterleri** '0'–'9'	'0' = 48 = 0x30
65–90	Büyük harfler A–Z	A = 65 = 0x41
97–122	Küçük harfler a–z	a = 97 = 0x61



Şekil 3.2: ASCII'nin 0–127 aralığının haritası. Büyük ve küçük harf arasındaki fark her zaman 32'dir.

Büyük–küçük harf farkı neden tam 32? $32 = 2^5$. A = 65 = 1000001₂ ve a = 97 = 1100001₂... dikkatli bak: aradaki tek fark 5 numaralı bit (ağırlığı 32). Tasarımcılar bunu bilerek yaptı: büyük/küçük harf çevirmek tek bir biti değiştirmek demek.

D1 s. 33'teki tabloyu okumak için: her sütun grubunda **Dec** (onluk), **Hex** (onaltılık), **Oct** (sekizlik) ve **Chr** (karakter) var. Örneğin "65 41 101 A" satırı: A'nın kodu onlukta 65, onaltılıkta 41, sekizlikte 101. Slaytın iki uyarısı:

- **Kontrol karakterlerinin görünür bir şekli (glyph) yoktur;** ekrana çizilmez, bir işi tetikler (satır atla, zil çal...).
- **Karakter kodu ile ekranda görünen yazı tipi (font) farklı kavramlardır.** 'A' harfi Arial'da da Times'ta da 65'tir; değişen yalnızca çizim.

▶ ÖRNEK 3.1 — Ek örnek: "Hi!" ASCII'de

Soru: "Hi!" metni ASCII ile nasıl saklanır? Onluk, onaltılık ve 8 bitlik ikilik yaz. Kaç bayt tutar?
Yöntem: Her karakterin kodunu tablodan bul; onaltılığı 16'ya bölerek, ikiliği hex rakamlarını 4 bite açarak elde et.

Karakter	Onluk	Hex	İkilik (8 bit)
H	72	48	0100 1000
i	105	69	0110 1001
!	33	21	0010 0001

Sonuç: 3 karakter → **3 bayt**: 48 69 21 (hex). ASCII 7 bitlik olsa da her karakter pratikte bir baytın içine konur; en soldaki bit 0 olur.
En sık hata: i'nin kodunu I ile karıştırmak (I = 73, i = 105; aradaki fark yine 32).

✗ SIK YAPILAN HATA — Rakam ile rakam karakteri

Yanlış: "Klavyeden 7 yazınca bellekte 0000 0111 saklanır." → **Doğrusu:** '7' karakteri ASCII 55 = 0x37 = 0011 0111 olarak saklanır. → **Neden:** Metin olarak girilen her şey karakter kodudur; '7' karakteri ile 7 sayısı farklı kodlama kuralları. D1 s. 62'deki örnekte de `char a` değişkeninde **0x30** duruyor ve slayt bunun "'0' karakteri" olduğunu not ediyor: 0x30 = 48 = '0'.

✎ DURUP DÜŞÜN

1. 'D' harfinin kodu 68 ise 'd', 'E' ve 'e'nin kodları nedir? Tablo kullanmadan bul.
2. ASCII'de Türkçe "ş" harfi var mı? Neden?

3.3 8 bitlik kod sayfaları ve "Ã§" karmaşası

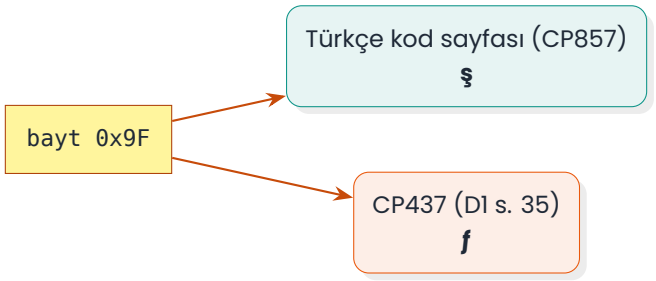
❖ Kaynak: D1 s. 34–35 / slayt 35

ASCII 7 bit kullanıyordu ama bilgisayarlar veriyi 8 bitlik baytlarla taşır. Boşta kalan bir bit, 128 yeni kod demek (128–255). Herkes bu boşluğu kendi diline göre doldurdu ve ortaya **kod sayfaları (code pages)** çıktı.

∑ TANIM / FORMÜL — Kod sayfası (8 bit, 1 bayt)

- **Ortak aralık:** İlk 128 kod (0–127) genelde ASCII ile aynıdır; temel Latin karakterler her yerde taşınabilir kalır.
- **Değişken aralık:** 128–255 arası kodlar **kod sayfasına göre değişir.**
- **Yanlış kod sayfası bozuk metin üretir.** (*"The wrong code page produces corrupted text."*)

D1 s. 35'teki "Extended ASCII" tablosunda 128 = Ç, 129 = ü, 130 = é... görüyorsun. Bu tablo aslında IBM PC'lerin **Code Page 437** tablosu; başka bir kod sayfasında 128'in yerinde bambaşka bir karakter durur. Tek bir "genişletilmiş ASCII" standardı yoktur.



Şekil 3.3: Aynı bayt, iki kod sayfasında iki farklı karakter. Metni yazan ile okuyan program aynı kod sayfasını kullanmazsa harfler bozulur.

+ EK BİLGİ (KAYNAK DIŞI) — Türkçe karakterler neden bozulur?

Türkçe için tarihte birkaç kod sayfası kullanıldı (DOS'ta CP857, Windows'ta Windows-1254, standart olarak ISO-8859-9). Bugün ise metinlerin çoğu UTF-8 ile saklanır. UTF-8 ile yazılmış "Ağaç" kelimesi Batı Avrupa kod sayfasıyla (Windows-1252) okunursa ekranda **"AÄŸaÃ§"** görürsün: ğ'nin iki baytı (C4 9F) iki ayrı Latin karakter sanılır. Bu bozulmaya halk arasında "mojibake" denir. Bu kitapta Python ile denendi.

3.4 EBCDIC: IBM'in kendi yolu

❖ Kaynak: D1 s. 36

∑ TANIM / FORMÜL — EBCDIC

- IBM tarafından **ana bilgisayar (mainframe)** sistemleri için geliştirildi.
- **8 bitlik** kod ailelerinden oluşur.
- Kod sıralaması **ASCII ile uyuşmaz.**
- Bazı mainframe ortamlarında ve eski (legacy) veri kümelerinde hâlâ karşına çıkar.

Pratik sonucu: EBCDIC dosyasını ASCII sanarak açarsan anlamsız karakterler görürsün; iki sistem arasında veri taşırken dönüştürme gerekir. (Açılımı Extended Binary Coded Decimal Interchange Code; sınavda büyük ihtimalle "IBM, mainframe, 8 bit, ASCII ile uyumsuz" dörtlüsü yeter.)

3.5 Unicode: Dünyanın bütün karakterlerine tek numara

❖ Kaynak: D1 s. 37–39 / slayt 37, 41, 38

Kod sayfası karmaşasının kökü şu: 256 kod, dünyanın bütün yazı sistemlerine yetmez ve her ülke aynı numaraları farklı harflere verdi. **Unicode** bu sorunu kökten çözmek için icat edildi: her karaktere, dünyada **tek ve değişmez** bir numara.

∑ TANIM / FORMÜL — Unicode ve kod noktası (code point)

- Unicode, İngilizceye ek olarak çok daha fazla dilin karakterlerini temsil etmek için icat edildi (D1 s. 37).
- Latin, Kiril, Arap, Han (Çin-Japon-Kore) ve birçok yazı sistemini; matematik sembollerini, okları ve emojileri kapsar (D1 s. 38).
- Her karakterin numarasına **kod noktası (code point)** denir ve **U+** önekiyle onaltılık yazılır:

A = **U+0041**.
- Ekranda görünmesi için yazı tipinde o karakterin **glifi (glyph)** de olmalıdır.

✘ SIK YAPILAN HATA — Slayttaki eski bilgi: "16 bit, 65.536 karakter yeter"

D1 s. 37'de Unicode'un 16 bitlik desen kullandığı ve 2^{16} = 65,536 kodun "dünyanın bütün dilleri için yeterli" olduğu yazıyor. Bu, Unicode'un **ilk sürümlerinin** (1990'ların başı) fikriydi; sonra yetmediği anlaşıldı.
Bugünkü durum: Unicode kod uzayı **U+0000'dan U+10FFFF'e** kadar, yani 1,114,112 kod noktası; bugün bunların 150 binden fazlası atanmış durumda (emojiler 65,536'nın dışında). Slaytın kendisi de bunu dolaylı olarak doğruluyor: D1 s. 41'deki tabloda UTF-16'nın bir karakter için **4 bayta** kadar çıkabildiği yazıyor; 16 bit yetseydi buna gerek olmazdı.
Aynı slayttaki "Microsoft ilk 256 sembolü kullanır" ifadesi de belirsiz: Windows bugün metni dahili olarak UTF-16 ile işler. "Java Unicode kullanır" kısmı doğru (Java'nın `char` türü 16 bitlik bir UTF-16 birimidir).
Sınavda ne yazmalı? Hoca slayttaki ifadeyi soruyorsa (ör. "Unicode kaç bit kullanır?") slaytın cevabı 16 bit / 65,536'dır; açık uçlu soruda bu cevabı verip "günümüzde Unicode U+10FFFF'e kadar genişletildi" notunu eklemen en güvenli yol. Bu nokta "Hocaya sorulacaklar" listesinde.

D1 s. 38'deki örnek karakterler (Python'un Unicode veritabanıyla kontrol edildi):

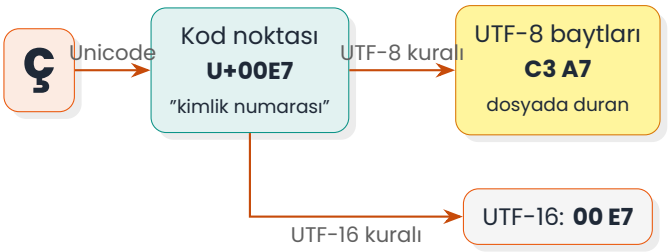
Kod noktası	Karakter	Kaynak	Not
U+0041	A	İngilizce (Latin)	ASCII 65 ile aynı numara
U+042F	Я	Rusça (Kiril)	
U+0E09	ฉ	Tay	Slaytta "OE09" (O harfiyle) yazılmış; doğrusu sıfır: 0E09
U+13EA	Ꮺ	Çeroki	
U+211E	℞	Harf benzeri semboller	"reçete" işareti
U+21CC	⇌	Oklar	
U+282F	⠯	Braille	
U+345F	㑟	Çince/Japonca/Korece	

Yazı tipinde glif yoksa ekranda içi boş bir kutu (□) görürsün: kod noktası doğru, ama çizecek şekil yok.

3.5.1 Kod noktası ≠ dosyadaki baytlar

❖ Kaynak: D1 s. 39 / slayt 38

Haftanın en önemli ayrımlarından biri bu:



Şekil 3.4: Unicode karakterin kimliğini (kod noktası) belirler; UTF kodlamaları bu kimliğin dosyada hangi baytlarla saklanacağını belirler (D1 s. 39 ve s. 66).

D1 s. 39: A'nın kod noktası **U+0041**, UTF-8'deki dosya temsili tek bayt **41**. A için ikisi aynı göründüğünden fark gözden kaçar; ç gibi ASCII dışındaki karakterlerde fark açıkça ortaya çıkar.

3.6 UTF-8, UTF-16, UTF-32: Aynı numarayı farklı paketlemek

❖ Kaynak: D1 s. 40–41 / slayt 39–40

∑ TANIM / FORMÜL — UTF kodlamaları

Kodlama	Kod birimi (code unit)	Karakter başına alan	Özellik
UTF-8	8 bit	1–4 bayt	ASCII uyumlu
UTF-16	16 bit	2 veya 4 bayt	Bazı platformlarda yaygın
UTF-32	32 bit	4 bayt	Sabit uzunluk, daha çok alan

UTF-8 (D1 s. 40): bir karakteri 1–4 baytla kodlar; ASCII karakterleri aynı değerle **tek baytta** saklar; değişken uzunluk Latin metinde yer kazandırır; web'de, dosya biçimlerinde ve yazılım arayüzlerinde yaygın kullanılır.

Benzetme: Kargo şirketi gibi düşün. UTF-32 her paketi (karakteri) aynı büyük kutuya koyar; işlemesi kolay ama çoğu kutu boş hava taşır. UTF-8 ise paketi boyutuna göre küçük, orta veya büyük kutuya koyar; yerden tasarruf eder ama alıcının her kutunun boyutunu okuyabilmesi gerekir. **Benzetmenin sınırı:** UTF-8'de "kutu boyu" bilgisi ayrı bir etikette değil, baytların ilk bitlerinin içinde gizlidir (aşağıdaki ek bilgi kutusu).

+ EK BİLGİ (KAYNAK DIŞI) — UTF-8 baytları nasıl oluşturur?

Slaytta yok; "1–4 bayt" ifadesinin arkasındaki kural şu:

Kod noktası aralığı	Bayt sayısı	Bit kalıbı
U+0000 – U+007F	1	0xxxxxxx
U+0080 – U+07FF	2	110xxxxx 10xxxxxx
U+0800 – U+FFFF	3	1110xxxx 10xxxxxx 10xxxxxx
U+10000 – U+10FFFF	4	11110xxx 10xxxxxx 10xxxxxx 10xxxxxx

ç = U+00E7: 0xE7 = 1110 0111₂, 11 bitle yaz: 00011 100111. Kalıba yerleştir: 110**00011** 10**100111** = C3 A7.
Türkçe özel harflerin (ç ğ ı ö ş ü ve büyükleri) hepsi UTF-8'de **2 bayt** tutar; Tay harfi ก ve ⇄ gibi semboller **3 bayt**, emojiler **4 bayt**. Hepsi Python ile doğrulandı.

▶ ÖRNEK 3.2 — Ek örnek: "Çağrı" kaç bayt?

Soru: "Çağrı" kelimesi UTF-8, UTF-16 ve UTF-32 ile kaç bayt tutar?
Verilenler: 5 karakter: Ç, a, ğ, r, ı. a ve r ASCII'de var; Ç, ğ, ı yok.
Yöntem: Karakter başına bayt sayısını kodlamaya göre topla.

Karakter	UTF-8	UTF-16	UTF-32
Ç	2	2	4
a	1	2	4
ğ	2	2	4
r	1	2	4
ı	2	2	4
Toplam	**8**	**10**	**20**

En sık hata: UTF-8'de her karakteri 1 bayt saymak (5 bayt). Bu yalnızca saf ASCII metinde doğrudur.
Başka nerede: Veritabanında "en fazla 255 karakter" alanının bayt olarak ne kadar yer tuttuğunu hesaplarken aynı mantık kullanılır.

▶ ÖRNEK 3.3 — Yarı çözümlü: "Merhaba"

"Merhaba" 7 karakter, hepsi ASCII.

- UTF-8: her karakter 1 bayt → 7 bayt
- UTF-16: her karakter … bayt → … bayt
- UTF-32: …

Tamamla. UTF-8 hangi metinlerde diğerlerine göre en çok yer kazandırır?

✎ KENDİN ÇÖZ

(a) ASCII'de "CMP" kelimesini onaltılık bayt dizisi olarak yaz (C = 67). (b) 'z' karakterinin kodu kaçtır? (c) "İş" kelimesi UTF-8 ve UTF-32 ile kaç bayttır? (d) Unicode kod noktası ile UTF-8 arasındaki farkı tek cümleyle açıkla.

Bağlantılar

- Bölüm 1: ASCII'nin 128 kodu 2^7'den, 8 bitlik kod sayfasının 256 kodu 2^8'den geliyor.
- Bölüm 2: ASCII tablosu onluk ve onaltılık sütunlarla okunur; Unicode kod noktaları onaltılıktır (U+0041 = 65).
- Bölüm 6: D1 s. 62'deki `char a = 0x30` bir karakter kodudur; değişkenler bellekte bu baytlarla durur.

⚠ SINAVDA DİKKAT

- ASCII: **7 bit, 128 kod** (0–127). A = 65 (0x41), a = 97 (0x61), '0' = 48 (0x30), boşluk = 32.
- 8 bitlik kod sayfaları: 0–127 ortak, **128–255 değişir**; yanlış sayfa = bozuk metin.
- EBCDIC: IBM mainframe, 8 bit, ASCII sırasıyla **uyumsuz**.
- **Unicode = karakter kimliği (kod noktası, U+XXXX); UTF = bayt temsili.** Bu ayrımı mutlaka kelimesi kelimesine yazabil.
- UTF-8: 1–4 bayt, ASCII uyumlu. UTF-16: 2 veya 4 bayt. UTF-32: sabit 4 bayt.

Kavram	Ne belirler?	Örnek (A)
Karakter kümesi / kod noktası	Hangi numara hangi karakter	U+0041

Kodlama (UTF-8...)	Numara dosyada hangi baytlarla durur	41
Yazı tipi (font)	Karakter ekranda nasıl çizilir	Arial'da A

❖ BU KONUDAN NASIL SORU GELİR?

1. Kavramsal: *"Explain the difference between a Unicode code point and UTF-8 encoding."* İpucu: "difference" → iki tanım + aynı karakter için örnek (ç: U+00E7 ↔ C3 A7).
2. Hesap: *"How many bytes does the string 'Ağaç' occupy in UTF-8 and in UTF-32?"* İpucu: ASCII karakterleri ayır (A, a → 1'er bayt), Türkçe harfler 2 bayt; UTF-32 her karakter 4.
3. Çoktan seçmeli: "Which encoding is ASCII-compatible and variable length?" → UTF-8. Çeldiriciler: UTF-32 (sabit), EBCDIC (ASCII ile uyumsuz).
4. Senaryo: *"A Turkish text file shows 'Ã§' instead of 'ç'. Explain why."* İpucu: dosya bir kodlamayla yazılmış, başka biriyle okunmuş; kod sayfası/kodlama uyuşmazlığı.

✗ SIK YAPILAN HATA

Yanlış: "ASCII 8 bittir, 256 karakter." → **Doğrusu:** ASCII 7 bit, 128 kod; 256'lık tablolar ASCII'nin üstüne kurulmuş 8 bitlik kod sayfalarıdır. → **Neden:** D1 s. 32 açıkça 7 bit diyor; 8. bitle gelen 128–255 aralığı sayfadan sayfaya değişir.
Yanlış: "Unicode bir dosya biçimidir / kodlamadır." → **Doğrusu:** Unicode karakterlere numara veren bir standarttır; dosyadaki baytları UTF-8/16/32 belirler. → **Neden:** Aynı U+00E7, UTF-8'de C3 A7, UTF-16'da 00 E7 olarak saklanır.
Yanlış: "UTF-16 her karakter için 2 bayttır." → **Doğrusu:** 2 **veya 4** bayt. → **Neden:** U+FFFF'in üstündeki karakterler (ör. emojiler) iki 16 bitlik birim gerektirir.

⚡ HIZLI TEKRAR

1. Metin, karakterlerin sayısal kodları olarak saklanır; şekiller değil.
2. ASCII: 7 bit, 128 kod; A 65, a 97, fark 32; kontrol karakterlerinin glifi yok.
3. 8 bit kod sayfaları: üst 128 kod ülkeye göre değişir → bozuk metin riski. EBCDIC: IBM'in ASCII'den farklı 8 bit kodu.
4. Unicode: her karaktere tek kod noktası (U+...). Glif yazı tipinden gelir.
5. UTF-8 (1–4, ASCII uyumlu), UTF-16 (2/4), UTF-32 (4, sabit).

Ezberle: A = 65, a = 97, '0' = 48, boşluk = 32; UTF tablosu. **Türetebil:** diğer harflerin kodları, metnin bayt boyutu.
Bu bölümden sonra şunları yapabiliyor olmalısın: kısa bir kelimeyi ASCII baytlarına çevirmek; kod noktası–kodlama ayrımını açıklamak; bir metnin UTF-8/16/32 boyutunu hesaplamak; bozuk karakterlerin nedenini anlatmak.

Bölüm 3 cevapları

Durup düşün 1. d = 68 + 32 = **100**, E = 69, e = 69 + 32 = **101**.

Durup düşün 2. Yok. ASCII yalnızca 128 kod tanımlar ve bunlar İngilizce harfler, rakamlar, noktalama ve kontrol karakterleri için kullanılmıştır. Türkçe harfler 8 bitlik kod sayfalarında (128–255 aralığında) ya da Unicode'da yer alır.

Yarı çözümlü. UTF-16: her karakter 2 bayt → 14 bayt; UTF-32: 4 · 7 = 28 bayt. UTF-8 en çok, ASCII

ağırlıklı (İngilizce, kod, Latin) metinlerde yer kazandırır.

Kendin çöz. (a) C = 67 = 0x43, M = 77 = 0x4D, P = 80 = 0x50 → **43 4D 50**. (b) Z = 90 → z = 90 + 32 = **122**. (c) İ ve ş ASCII dışında, her biri UTF-8'de 2 bayt → **4 bayt**; UTF-32: 2 · 4 = **8 bayt**. (d) Kod noktası karakterin Unicode'daki kimlik numarasıdır (ç = U+00E7); UTF-8 bu numaranın dosyada hangi bayt dizisiyle saklanacağını belirleyen kuraldır (C3 A7).

BÖLÜM 4

Görüntünün Temsili

◆ **BU BÖLÜMDE** Okuma süresi: 55 dk Zorluk: ●●○

- Bit eşlem (bitmap) görüntünün piksel matrisi olarak saklanmasını
- Çözünürlük ve renk derinliğini; b bit/piksel → 2^b renk kuralını
- RGB modelini ve #RRGGBB onaltılık renk yazımını
- Sıkıştırılmamış görüntü boyutunu bit → bayt → MiB adımlarıyla hesaplamayı
- Paletli (dizinli) renk ve vektör grafik fikirlerini, bitmap ile karşılaştırmayı

SINAV ÖNCELİĞİ: YÜKSEK ❖ Kaynak: D1 s. 42–55 / slayt 45–57

Sınav önceliği yüksek: "Calculate digital media sizes from basic parameters" bir öğrenme çıktısı ve slaytta iki tam çözümlü boyut hesabı var (s. 50, 52). Hesap sorusu ve bitmap–vektör karşılaştırması bekle.

→ BU KONUYA NEDEN GEÇİYORUZ?

Metinde her karaktere bir numara veriyorduk. Bir fotoğrafta "karakter" yok; sonsuz ton ve şekil var. Çözüm, resmi küçük karelere bölüp her kareye bir **renk numarası** vermek. Bölüm 1'deki 2^n kuralı burada "kaç renk?" sorusunu, Bölüm 2'deki onaltılık sistem de #FF0000 gibi renk kodlarını açıklayacak. Bu bölümün sonunda telefonundaki bir fotoğrafın neden megabaytlarca yer tuttuğunu kendin hesaplayabileceksin.

4.1 Bir resim nasıl sayılara dönüşür?

❖ Kaynak: D1 s. 43 / slayt 45

∑ TANIM / FORMÜL — Piksel matrisi

- **Bit eşlem görüntü (bitmap / raster image):** satır ve sütunlara dizilmiş **piksellerden** oluşan bir matris.
- Her **piksel** bir renk değeri saklar.
- **Çözünürlük (resolution):** yatay ve dikey piksel sayısı (ör. 1920 × 1080).
- **Renk derinliği (color depth):** bir piksel için kullanılan bit sayısı; birimi **bpp** (bits per pixel).

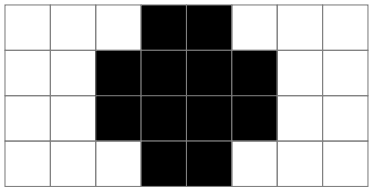
Sıkıştırılmamış boyut ≈ genişlik × yükseklik × piksel başına bit (bit cinsinden)

Formülün nedeni sade: Resimde genişlik × yükseklik kadar piksel var; her biri aynı sayıda bit tutuyor; toplam bit = piksel sayısı × piksel başına bit. "Yaklaşık" denmesinin sebebi, gerçek dosyalarda başlık bilgisi, satır dolgusu ve en önemlisi **sıkıştırma** (JPEG, PNG) olması.

4.2 Siyah-beyaz: piksel başına tek bit

❖ Kaynak: D1 s. 44–45 / slayt 46–47

En basit durumda her pikselin iki hâli var: siyah ya da beyaz. İki durum için **1 bit** yeter. Bir bayt 8 bit olduğundan **8 piksel 1 bayta** sığar.



```
00011000
00111100
00111100
00011000
```

Görüntü Matris gösterimi

```
00011000 00111100 00111100 00011000
```

Doğrusal gösterim: bellekteki sıra

Şekil 4.1: D1 s. 45: 4 × 8 piksellik iki renkli görüntü önce matris olarak düşünülür, sonra satırlar art arda eklenerek doğrusal bir bit dizisi olarak saklanır. 32 piksel = 32 bit = 4 bayt.

D1 s. 45'in mesajları:

- İki boyutlu piksel matrisi, bitlerin ya da baytların **doğrusal bir dizisi** olarak saklanır (bellek tek boyutludur, Bölüm 6).
- Satır sırası ve satır dolgusu (**row padding**) dosya biçimine göre değişebilir.
- Görüntüleyici program, matrisi yeniden kurmak için **boyutları** (genişlik, yükseklik) kullanır.

Son madde önemli: 32 bitlik diziye bakan program, genişliği bilmezse bunu 4 × 8 mi, 8 × 4 mü, 2 × 16 mı çizeceğini bilemez. Boyutlar dosyanın başlığında saklanır.

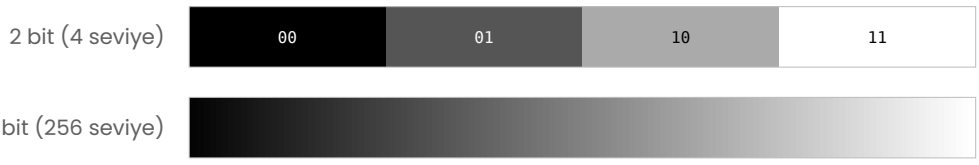
✗ SIK YAPILAN HATA — Slaytlar arasındaki çelişki: siyah 0 mı, 1 mi?

D1 s. 44 "0 siyah, 1 beyaz" diyor. Ama D1 s. 45'teki şekilde **siyah kareler 1** ile gösterilmiş (ilk satırdaki iki siyah kare matriste **1** olarak yazılmış: 000**11**000). İki slayt ters kural kullanıyor. **Doğrusu:** Hangisinin siyah olduğu bir **kodlama kuralı** seçimidir (Bölüm 1: bitlerin anlamını kural belirler); dosya biçimine göre değişir. Sınavda soruda hangi kural verilmişse onu kullan; verilmemişse varsayımını açıkça yaz ("0 = beyaz, 1 = siyah kabul edersek..."). Bu nokta "Hocaya sorulacaklar"da.

4.3 Gri tonlama ve renk derinliği

❖ Kaynak: D1 s. 46–47 / slayt 48–49

Siyah ile beyazın arasına ara tonlar eklemek için piksel başına daha çok bit gerekir. 2 bitle 4 gri seviyesi, 8 bitle 256 gri seviyesi elde edilir; seviye arttıkça ton geçişleri yumuşar (D1 s. 46).



Şekil 4.2: 2 bitle yalnızca 4 basamaklı gri tonu (burada 00 = siyah kabul edildi), 8 bitle ise gözün ayırt edemeyeceği kadar yumuşak bir geçiş elde edilir.

∑ TANIM / FORMÜL — b bit/piksel → 2^b renk

Renk derinliği	1 bpp	2 bpp	8 bpp	24 bpp
Renk sayısı	2	4	256	16,777,216

Bu, Bölüm 1'deki n bit → 2^n kuralının ta kendisi. Daha yüksek renk derinliği hem **dosya boyutunu** hem **renk ayrıntısını** artırır (D1 s. 47).

4.4 RGB: Ekran renkleri ışıkla karıştırır

❖ Kaynak: D1 s. 48–49 / slayt 50–51

Telefon ekranına büyüteçle bakarsan her pikselin aslında kırmızı, yeşil ve mavi üç küçük ışık kaynağından oluştuğunu görürsün. Ekran rengi bu üç ışığın şiddetlerini ayarlayarak üretir.

∑ TANIM / FORMÜL — RGB renk modeli

- Ekranlar rengi **kırmızı (R), yeşil (G) ve mavi (B)** ışık bileşenlerinden oluşturur.
- Yaygın **24 bit RGB**, her kanal için **8 bit** kullanır (3 × 8 = 24).
- Her kanal **0–255** arası bir şiddet değeri alır (8 bit → 2^8 = 256 seviye).
- R = 255, G = 255, B = 255 **beyaz**; her kanal 0 **siyah**.
- Onaltılık yazım: her kanal iki hex rakam: **#RRGGBB**.



Şekil 4.3: D1 s. 49'daki RGB tablosu, gerçek renkleriyle ve onaltılık kodlarıyla. Slaytın "purple" (mor) dediği (157, 95, 82) ekranda kırmızımsı-kahverengi görünür; renk adları yaklaşıktır.

D1 s. 49'un notları: **sarı = yüksek kırmızı + yüksek yeşil** (ışıkta kırmızı + yeşil = sarı; boyada öyle değildir!), pembe ve kahverengi gibi renkler kanalların farklı şiddetleriyle elde edilir.

▶ ÖRNEK 4.1 — Ek örnek: RGB ↔ #RRGGBB

Soru 1: Kahverengi (146, 81, 0) için hex kodu bul.
146 ÷ 16 = 9 kalan 2 → **92**; 81 ÷ 16 = 5 kalan 1 → **51**; 0 → **00**. Sonuç: **#925100**.
Soru 2: #1E90FF hangi RGB değerleridir?
1E = 1 · 16 + 14 = **30**; 90 = 9 · 16 + 0 = **144**; FF = **255**. Sonuç: (30, 144, 255): mavisi baskın, açık bir mavi.
En sık hata: Kanalı tek hex rakamla yazmak (0 → "0" yerine **"00"**). #RRGGBB'de her kanal **tam iki** rakamdır, aksi hâlde kanallar kayar.

4.5 Çözünürlük ve boyut hesabı

❖ Kaynak: D1 s. 50–52 / slayt 52–54

Aynı alanı daha az pikselle göstermek ayrıntı kaybettirir (D1 s. 51'deki köpek fotoğrafı: solda çok

piksel, sağda bloklar hâlinde az piksel). Ama her piksel bit demek; çözünürlük arttıkça dosya büyür.

↺ HATIRLATMA — Bit, bayt, KiB, MiB

- 1 bayt = 8 bit → bit'ten bayta geçmek için **8'e böl**.
- 1 KiB = 2^{10} = 1,024 bayt; 1 MiB = 2^{20} = 1,048,576 bayt; 1 GiB = 2^{30} bayt.
- Bayttan MiB'a geçmek için **1,048,576'ya böl** (ya da 1024'e iki kez).

+ EK BİLGİ (KAYNAK DIŞI) — MB mi, MiB mi?

Slaytlar **MiB, GiB** (ikili önekler, 2'nin kuvvetleri) kullanıyor. Günlük hayatta görülen **MB, GB** ise standart olarak 10'un kuvvetleridir: 1 MB = 1,000,000 bayt. Bu yüzden "1 TB" diye satılan disk bilgisayarda yaklaşık 931 GiB görünür. Sınavda hangi birim istenmişse ona çevir; slaytlar ikili önekleri kullandığı için varsayılan MiB.

▶ ÖRNEK 4.2 — Slayt örneği [D1 s. 52 / slayt 54]: 800 × 600, 24 bpp

Verilenler: genişlik 800, yükseklik 600, renk derinliği 24 bit/piksel. **İstenen:** sıkıştırılmamış boyut (MiB).
Yöntem: piksel sayısı → bit → bayt → MiB. Her adımda birimi yaz.

1. Piksel: 800 × 600 = **480,000 piksel**
2. Bit: 480,000 × 24 = **11,520,000 bit**
3. Bayt: 11,520,000 ÷ 8 = **1,440,000 bayt**
4. MiB: 1,440,000 ÷ 1,048,576 ≈ **1.37 MiB**

Kısayol: 24 bit = 3 bayt olduğundan 480,000 × 3 = 1,440,000 bayt doğrudan.
En sık hata: 8'e bölmeyi unutup 11,520,000'i bayt sanmak (sonuç 8 kat büyük çıkar) ya da 1,000,000'a bölüp 1.44 "MiB" yazmak (o MB olur).

▶ ÖRNEK 4.3 — Slayt örneği [D1 s. 50 / slayt 52]: Full HD, 24 bit

1920 × 1080 = **2,073,600 piksel** → × 24 = **49,766,400 bit** → ÷ 8 = **6,220,800 bayt** → ÷ 1,048,576 ≈ **5.93 MiB**.
Tek bir Full HD karenin sıkıştırılmamış hâli yaklaşık 6 MiB. Bunu aklında tut, Bölüm 5'te videoda saniyede 30 tane olacak.

▶ ÖRNEK 4.4 — Yarı çözümlü: 1024 × 768, 8 bit gri tonlama

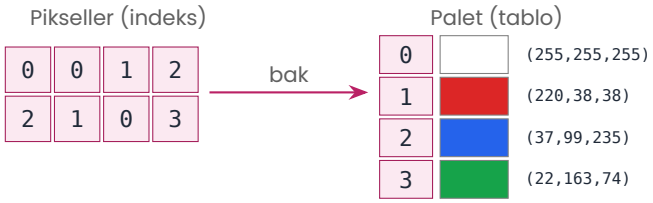
1. Piksel: 1024 × 768 = 786,432
2. Bit: 786,432 × 8 = ...
3. Bayt: ... (ipucu: 8 bit = 1 bayt olduğundan 2. ve 3. adım birbirini götürür)
4. KiB ve MiB: ... ÷ 1024 = ... KiB = ... MiB

4.6 Palet: Renk yerine "renk numarası" saklamak

❖ Kaynak: D1 s. 53 / slayt 55

Bir logoda belki sadece 10 renk var. Her pikselde 24 bitlik tam RGB değeri saklamak israf. Bunun yerine bir **palet (palette)** hazırlanır: renklerin listesi. Her piksel, rengin kendisi yerine **paletteki sıra**

numarasını (indeks) saklar.



Şekil 4.4: Dizinli renk: piksel "1" diyorsa görüntüleyici paletin 1 numaralı satırına bakar ve oradaki RGB rengini çizer.

∑ TANIM / FORMÜL — Palet ve renk indeksi (indexed color)

- Belirli bir uygulama (ör. bir tarayıcı) yalnızca sınırlı sayıda belirli rengi destekleyerek bir **renk paleti** oluşturabilir. Slayttaki örnek: Netscape Navigator'ın paleti.
- Piksel, doğrudan RGB değeri yerine **palet girişinin numarasını** saklar.
- **8 bitlik indeks en fazla 256** palet rengini seçebilir (2^8).
- Az renkli grafiklerde dosya boyutunu küçültebilir.

▶ ÖRNEK 4.5 — Ek örnek: Palet ne kadar kazandırır?

800 × 600'lük bir görüntü 24 bpp ile 1,440,000 bayt (slayt 54). Aynı görüntü 256 renklik paletle saklanırsa: her piksel 8 bit = 1 bayt → **480,000 bayt**, artı paletin kendisi 256 × 3 bayt = **768 bayt**. Toplam 480,768 bayt, yani yaklaşık **üçte bir**. Bedeli: görüntüde en fazla 256 farklı renk olabilir; fotoğraf için yetmez.

4.7 Vektör grafik: Piksel yerine tarif saklamak

❖ Kaynak: D1 s. 54–55 / slayt 56–57

Bir daireyi binlerce piksel olarak saklamak yerine "merkezi (50, 50), yarıçapı 30, rengi mavi" diye **tarifini** saklayabilirsin. Vektör grafik budur.

∑ TANIM / FORMÜL — Vektör grafik

- Görüntü çizgi, eğri, dolgu ve metin gibi **geometrik nesnelerden** oluşur.
- Nesneler **koordinatlar ve matematiksel parametrelerle** saklanır: bir çizgi uç noktalarının koordinatlarıyla, bir daire **merkez koordinatları ve yarıçapıyla** tanımlanır.
- Avantajı: formül bir ölçek çarpanıyla çarpılarak **çözünürlükten bağımsız** büyütülebilir.
- Büyütmede kenarlar keskin kalır, çünkü sistem nesneleri **yeniden hesaplar**.



Şekil 4.5: Bitmap büyütülünce pikseller kare kare görünür; vektör daire yeniden hesaplanır ve kenarı keskin kalır. Sağda dosyada saklanan tarif.

	Bitmap	Vektör
Neyden oluşur	Pikseller	Geometrik nesneler
Neye uygun	Fotoğraflar, dokular	Logolar, ikonlar, çizimler
Büyütünce	Ayrıntı yoksa bloklaşır	Keskin kalır (yeniden hesaplanır)
Boyutu neye bağlı	Çözünürlük × renk derinliği	Nesne sayısı ve karmaşıklığı
Örnek biçimler	BMP, PNG, JPEG	SVG, PDF içindeki çizimler

Slaytın son cümlesi: *"Neither format is always superior. The content determines the choice."* İçerik seçimi belirler: bir fotoğrafı vektörle tarif etmek neredeyse imkânsız, bir logoyu bitmap saklamak ise büyütmede kalite kaybı demek.

★ MERAKLISINA (SINAV DIŞI)

Bu kitaptaki bütün şekiller (kutular, oklar, renk örnekleri) vektör olarak çizildi. PDF'i istediğin kadar yakınlaştır: kenarlar hiç bozulmaz. Slaytlardaki fotoğraflar ise bitmap; yakınlaştırınca pikselleri görürsün.

✎ KENDİN ÇÖZ

(a) 640 × 480 boyutlu, 24 bpp bir görüntünün sıkıştırılmamış boyutunu bayt ve KiB olarak bul. (b) Aynı görüntü 16 renklik bir paletle saklansaydı piksel başına kaç bit gerekirdi ve piksel verisi kaç bayt olurdu? (c) #00FF00 hangi renktir, RGB değerlerini yaz.

✎ DURUP DÜŞÜN

1. Bir görüntünün çözünürlüğünü hem yatayda hem dikeyde iki katına çıkarırsan sıkıştırılmamış boyut kaç katına çıkar?
2. 24 bpp'den 32 bpp'ye geçen bir sistemde (çoğu zaman 8 bit saydamlık eklenir) renk sayısı değişir mi?

Bağlantılar

- **2^n kuralı** (Bölüm 1) → 2^b renk; 8 bit kanal → 0–255.
- **Onaltılık** (Bölüm 2) → #RRGGBB, FF = 255.
- **Boyut formülü** Bölüm 5'te kare hızı ve süreyle çarpılarak video boyutuna dönüşecek.
- **Doğrusal saklama** fikri Bölüm 6'daki tek boyutlu bellek ve ardışık adreslerle aynı.

⚠ SINAVDA DİKKAT

- Boyut (bit) = genişlik × yükseklik × bpp; **bayt için 8'e böl**, MiB için 1,048,576'ya böl.
- b bpp → **2^b** renk; 24 bit RGB → 16,777,216 renk.
- RGB: (0,0,0) siyah, (255,255,255) beyaz; #RRGGBB'de her kanal 2 hex rakam.
- Palet: piksel renk değil **indeks** saklar; 8 bit indeks → 256 renk.
- Bitmap = piksel (fotoğraf), vektör = geometrik nesne (logo); vektör büyütmede keskin kalır.

Karıştırılan	Fark
Çözünürlük	Kaç piksel (genişlik × yükseklik)
Renk derinliği	Her piksel kaç bit (→ kaç renk)

❖ BU KONUDAN NASIL SORU GELİR?

1. Hesap (kesin gibi): *"Calculate the uncompressed size of a 1280 × 720 image with 24-bit color in bytes and MiB."* İpucu: dört adımı yaz, birimi her satırda belirt.
2. Kavramsal karşılaştırma: *"Compare bitmap and vector graphics. Which is suitable for a company logo? Why?"* İpucu: tabloyu yaz, gerekçe: ölçekleme ve keskinlik.
3. Ters soru: "Bir görüntünün piksel verisi 3,000 bayt, boyutu 100 × 30. Renk derinliği nedir?" İpucu: formülü ters çevir: bpp = bit / piksel = 24,000 / 3,000 = 8.
4. Çoktan seçmeli: "How many colors can be represented with 4 bits per pixel?" → 16. Çeldiriciler: 4, 8, 256.

✘ SIK YAPILAN HATA

Yanlış: Bitleri bayt sanmak (800 × 600 × 24 = 11,520,000 "bayt"). → **Doğrusu:** 11,520,000 bit = 1,440,000 bayt. → **Neden:** bpp **bit** cinsindendir; bayta geçmek için 8'e böl.
Yanlış: "Renk derinliği 24 ise 24 renk vardır." → **Doğrusu:** 2^{24} = 16,777,216 renk. → **Neden:** Bit sayısı değil, bitlerin oluşturabileceği desen sayısı renk sayısını verir.
Yanlış: "Çözünürlüğü iki katına çıkarmak boyutu iki katına çıkarır." → **Doğrusu:** Her iki boyut ikiye katlanırsa piksel sayısı 4 katına çıkar. → **Neden:** Boyut genişlik ile yüksekliğin **çarpımıyla** orantılı.

⚡ HIZLI TEKRAR

1. Bitmap = piksel matrisi; bellekte satır satır doğrusal saklanır; boyutlar matrisi geri kurar.
2. Renk derinliği b → 2^b renk; 1 bit siyah-beyaz, 8 bit 256 gri ton, 24 bit RGB.
3. RGB: 3 kanal × 8 bit, 0–255; #RRGGBB.
4. Boyut = G × Y × bpp bit → ÷ 8 bayt → ÷ 2^{20} MiB.
5. Palet: indeks saklar, az renkli grafikte yer kazandırır.
6. Vektör: nesne tarifleri, ölçeklenince keskin; içerik seçimi belirler.

Ezberle: 2^{24} = 16,777,216; 1 MiB = 1,048,576 bayt. **Türetebil:** boyut formülü, RGB ↔ hex.
Bu bölümden sonra şunları yapabiliyor olmalısın: herhangi bir görüntünün sıkıştırılmamış boyutunu hesaplamak; RGB ile hex renk arasında çeviri yapmak; bitmap ile vektörü karşılaştırmak.

Bölüm 4 cevapları

Durup düşün 1. 4 katına. Piksel sayısı (2G) × (2Y) = 4 · G · Y.

Durup düşün 2. Fazladan 8 bit bir **saydamlık (alfa)** kanalı için kullanılıyorsa görünen renk sayısı yine 2^{24}'tür; her renk farklı saydamlık seviyelerinde gösterilebilir. Bitler sadece 2^b'ye göre ayrı desen sayısını söyler; bu desenlerin neyi kodladığını yine kural belirler.

Yarı çözümlü. 2. 6,291,456 bit; 3. 786,432 bayt; 4. 786,432 ÷ 1024 = **768 KiB** = 768 ÷ 1024 = **0.75**

MiB.

Kendin çöz. (a) 640 × 480 = 307,200 piksel × 24 = 7,372,800 bit ÷ 8 = **921,600 bayt** ÷ 1024 = **900 KiB** (≈ 0.88 MiB). (b) 16 = 2^4 → **4 bit/piksel**; 307,200 × 4 = 1,228,800 bit ÷ 8 = **153,600 bayt** (artı palet). (c) R = 0, G = 255, B = 0 → saf **yeşil**.

BÖLÜM 5

Video ve Ses

◆ **BU BÖLÜMDE** Okuma süresi: 35 dk Zorluk: ●●○

- Videonun zaman sırasıyla gösterilen görüntü karelerinden oluştuğunu ve kare hızını
- Sıkıştırılmamış video veri hızını adım adım hesaplamayı (D1 s. 58)
- Analog sesin örnekleme, nicemleme ve kodlama ile sayısala dönüşmesini
- Ses dosyası boyutunu örnekleme hızı, bit derinliği, kanal sayısı ve süreden hesaplamayı

SINAV ÖNCELİĞİ: ORTA ❖ Kaynak: D1 s. 56–59 / slayt 59–62

Sınav önceliği orta: yalnızca üç içerik slaytı var ama video hesabı Bölüm 4'teki boyut formülünün doğal devamı ve "Media size grows with resolution, color or sample bits, rate, and duration" özet slaytında (D1 s. 66) geçiyor. Bir hesap ve bir "örnekleme/nicemleme nedir?" sorusu gelebilir.

→ BU KONUYA NEDEN GEÇİYORUZ?

Bölüm 4'te tek bir resmin boyutunu hesapladık. Ama elimizdeki araçlar **zamanla değişen** şeyleri anlatamıyor: hareket eden görüntü ve havada titreşen ses. Video için cevap kolay: resimleri art arda, hızla göstermek. Ses ise daha ilginç; sürekli değişen bir dalgayı sonlu sayıda sayıya çevirmek zorundayız. Bu bölümde iki yeni çarpan öğreniyoruz: **hız** (saniyede kaç kare / örnek) ve **süre**.

5.1 Video: saniyede 30 fotoğraf

❖ Kaynak: D1 s. 57–58 / slayt 59–60

Defterinin köşesine her sayfada biraz farklı bir çöp adam çizip sayfaları hızla çevirdiysen video ilkesini zaten biliyorsun. Göz, yeterince hızlı değişen kareleri akıcı hareket olarak algılar.

∑ TANIM / FORMÜL — Video ve kare hızı

- Video, **zaman sırasıyla** gösterilen görüntü **karelerinden (frames)** oluşur.
- **Kare hızı (frame rate)** saniyedeki kare sayısıyla ölçülür: **fps** (frames per second).
- Yüksek kare hızı **daha akıcı** hareket gösterir ve **veri miktarını artırır**.
- Video genellikle görüntüleri, sesi, zamanlama bilgisini ve **üst veriyi (metadata)** birlikte taşır.

Sıkıştırılmamış video veri hızı (bit/s) = genişlik × yükseklik × bpp × fps
Boyut (bit) = veri hızı × süre (s)

Formül Bölüm 4'teki kare boyutunu fps ile çarpmaktan ibaret: bir kare G × Y × bpp bit ise, saniyede fps tanesi gösterildiğine göre saniyede G × Y × bpp × fps bit akar.

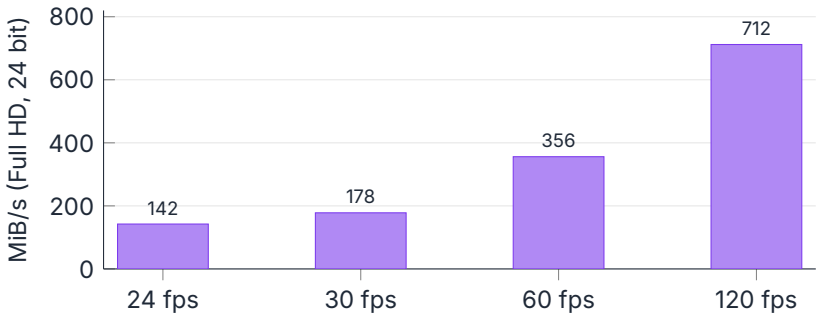
▶ ÖRNEK 5.1 — Slayt örneği [D1 s. 58 / slayt 60]: 1920 × 1080, 24 bit, 30 fps

Verilenler: G = 1920, Y = 1080, bpp = 24, fps = 30. **İstenen:** saniyedeki bit, bayt, MiB; 1 dakikalık boyut (GiB).

1. Bir kare: 1920 × 1080 × 24 = 49,766,400 bit (Bölüm 4)
2. Saniyede: 49,766,400 × 30 = **1,492,992,000 bit/s**
3. Bayt: 1,492,992,000 ÷ 8 = **186,624,000 bayt/s**
4. MiB: 186,624,000 ÷ 1,048,576 ≈ **178 MiB/s** (tam değer 177.98)
5. Bir dakika: 186,624,000 × 60 = 11,197,440,000 bayt ÷ 2^{30} (1,073,741,824) ≈ **10.4 GiB**

Yorum: Sıkıştırılmamış Full HD video dakikada 10 GiB'tan fazla! Telefonundaki bir dakikalık videonun birkaç yüz MB olması, video sıkıştırmanın (codec'lerin, örneğin H.264) ne kadar güçlü olduğunu gösteriyor. Ayrıca bu hesapta ses yok; slayt da yalnızca görüntü verisini hesaplıyor.

En sık hata: GiB'a geçerken 1,048,576'ya bölüp durmak (sonuç MiB olur) ya da 1,000,000,000'a bölmek (GB olur).



Şekil 5.1: Sıkıştırılmamış veri hızı kare hızıyla doğru orantılı artar: fps iki katına çıkınca veri de iki katına çıkar.

5.2 Ses: Sürekli dalgadan sayılara

❖ Kaynak: D1 s. 59 / slayt 62

Mikrofon havadaki basınç değişimini sürekli değişen bir **analog sinyale** çevirir. "Sürekli" demek, iki an arasında her zaman başka bir an ve başka bir değer olması demek: sonsuz bilgi. Bilgisayar sonsuzu saklayamaz; iki şeyi sonlu hâle getirmeliyiz: **zamanı** ve **değeri**.



Şekil 5.2: Turkuaz: analog sinyal. Mavi noktalar: **örnekleme** anlarında ölçülen gerçek değerler. Kırmızı çubuklar: **nicemleme** sonrası en yakın tam seviyeye yuvarlanmış değerler (3 bit $\rightarrow 2^3 = 8$ seviye: 0–7). Son adımda her seviye 3 bitlik koda çevrilir: 4 → 100, 7 → 111...

∑ TANIM / FORMÜL — Analogdan sayısala ses: üç adım

1. **Örnekleme (sampling):** Sinyal belirli anlarda ölçülür. *"Sampling measures the signal at specific moments."* Saniyedeki ölçüm sayısı **örnekleme hızı (sampling rate)**, birimi Hz.
2. **Nicemleme (quantization):** Her ölçüm, **sonlu sayıda** sayısal seviyeden birine eşlenir. *"Quantization maps each measurement to one of a finite number of digital levels."* b bit kullanılırsa 2^b seviye vardır; buna **bit derinliği (bit depth)** denir.
3. **Kodlama (coding):** Her seviye numarası ikilik koda çevrilir ve bit dizisi oluşur.

D1 s. 59'daki şema da bu üç kutuyu gösteriyor: kırmızı dalga → **Sampling** → çubuklar → **Quantization** → tam sayılar (4, 6, 4, ..., 25, ..., 15) → **Coding** → 00000100 ... 00001111. Son satırdaki 00000100 = 4 ve 00001111 = 15: şemada her örnek 8 bitlik kodla yazılmış.

Kayıp nerede? Örnekleme iki ölçüm arasındaki değişimi kaçırabilir; nicemleme gerçek değeri en yakın seviyeye yuvarlar (şekilde 1. örnekte 6.64 → 7). Örnekleme hızını ve bit derinliğini artırmak kaybı azaltır ama veri miktarını büyütür. Görüntüdeki çözünürlük–renk derinliği ilişkisinin aynısı:

Görüntü	Ses	Neyi belirler
Çözünürlük (piksel sayısı)	Örnekleme hızı (saniyedeki örnek)	"Ne kadar sık" bakıldığı
Renk derinliği (bpp)	Bit derinliği (bit/örnek)	Her ölçümün ne kadar ince ayrıldığı

Slayt ses boyutu için formül vermiyor, ama özet slaytı (D1 s. 66) boyutun "örnek bitleri, hız ve süre" ile büyüdüğünü söylüyor. Video formülüyle aynı mantıkla:

∑ TANIM / FORMÜL — Sıkıştırılmamış ses boyutu

Boyut (bit) = örnekleme hızı (örnek/s) × bit derinliği (bit/örnek) × kanal sayısı × süre (s)
Kanal: mono = 1, stereo = 2 (sol ve sağ hoparlör için ayrı örnekler).

+ EK BİLGİ (KAYNAK DIŞI) — Müzik CD'si

Müzik CD'leri saniyede **44,100** örnek, **16 bit** derinlik ve **stereo** kullanır. 44,100 × 16 × 2 = **1,411,200 bit/s** = 176,400 bayt/s. Bir dakika: 176,400 × 60 = 10,584,000 bayt ≈ **10.09 MiB**. MP3 gibi biçimler bunu yaklaşık 10 kat küçültür. (Bu sayılar slaytlarda yok.)

▶ ÖRNEK 5.2 — Ek örnek: Kısa bir HD video klip

Soru: 1280 × 720, 24 bit renk, 25 fps, 10 saniyelik sıkıştırılmamış videonun (sessiz) boyutu kaç MiB?

1. Bir kare: 1280 × 720 = 921,600 piksel × 3 bayt = 2,764,800 bayt
2. Saniyede: 2,764,800 × 25 = 69,120,000 bayt/s
3. 10 saniye: 691,200,000 bayt
4. MiB: 691,200,000 ÷ 1,048,576 ≈ **659.2 MiB**

Neden 3 bayt ile çarptık: 24 bit = 3 bayt; bit → bayt dönüşümünü baştan yapmak sayıları küçük tutar.

▶ ÖRNEK 5.3 — Yarı çözümlü: 1 dakikalık telefon kalitesinde ses

Örnekleme hızı 8,000 Hz, bit derinliği 8 bit, mono, süre 60 s.

1. Saniyede bit: 8,000 × 8 × 1 = ...
2. 60 saniyede bit: ...
3. Bayt: ... ÷ 8 = ...
4. KiB: ... ÷ 1024 ≈ ...

✎ KENDİN ÇÖZ

(a) Bir ses kaydında nicemleme için 4 bit kullanılıyor. Kaç farklı seviye var? (b) 48,000 Hz, 24 bit, stereo, 2 dakikalık sıkıştırılmamış ses kaç bayttır? (c) Bir videonun kare hızını 30'dan 60 fps'e çıkarırsan sıkıştırılmamış veri hızı nasıl değişir?

✎ DURUP DÜŞÜN

1. Örnekleme hızını çok düşük seçersen (ör. saniyede 10 örnek) bir şarkıya ne olur?

★ MERAKLISINA (SINAV DIŞI)

Neden CD'de tam 44,100 örnek? İnsan kulağı yaklaşık 20,000 Hz'e kadar sesleri duyar ve bir dalgayı doğru yakalamak için en yüksek frekansının **en az iki katı** hızda örneklemek gerekir (Nyquist–Shannon örnekleme teoremi; ileride "Sinyaller ve Sistemler" dersinde göreceksin). 2 × 20,000 = 40,000'in biraz üstü güvenlik payıyla seçildi.

Bağlantılar

- Video = Bölüm 4'teki görüntü boyutu × fps × süre.
- Nicemlemede b bit → 2^b seviye: Bölüm 1'deki 2^n kuralının ses versiyonu.
- Örnekleme + nicemleme, Bölüm 1'deki "algılama → sayısallaştırma → bit dizisi" zincirinin ses için açılmış hâli.

⚠ SINAVDA DİKKAT

- Video veri hızı (bit/s) = G × Y × bpp × **fps**; boyut = hız × **süre**.
- Örnekleme = **zamanda** ayrıklaştırma (ne zaman ölçülür); nicemleme = **değerde** ayrıklaştırma (hangi seviyeye yuvarlanır). Karıştırma!
- Ses boyutu = örnekleme hızı × bit derinliği × kanal × süre.
- Birim zinciri: bit → ÷8 bayt → ÷2^{20} MiB → ÷2^{10} GiB.

Kavram	Soru	Birim
Örnekleme hızı	Saniyede kaç ölçüm?	Hz (örnek/s)
Bit derinliği	Her ölçüm kaç bit (kaç seviye)?	bit/örnek
Kare hızı	Saniyede kaç kare?	fps

❖ BU KONUDAN NASIL SORU GELİR?

1. Hesap: *"Calculate the data rate of uncompressed video: 1280 × 720, 24-bit color, 30 fps. Give the result in bytes per second and MiB/s."* İpucu: kare boyutu × fps; birimleri satır satır yaz.
2. Kavramsal: *"Explain sampling and quantization in analog-to-digital audio conversion."* İpucu: biri zaman, biri değer ekseni; her biri için bir cümle tanım + "neyi artırınca kalite artar" bilgisi.
3. Çoktan seçmeli tuzak: "Which parameter does NOT affect the uncompressed size of a video?" Şıklarda çözünürlük, fps, süre, renk derinliği ve **ekran parlaklığı** gibi alakasız bir seçenek bulunur.

✘ SIK YAPILAN HATA

Yanlış: "Nicemleme, sinyalin belirli anlarda ölçülmesidir." → **Doğrusu:** Bu örneklemedir; nicemleme ölçülen değeri sonlu seviyelerden birine yuvarlamaktır. → **Neden:** İki adım iki farklı ekseni sonlu hâle getirir: zaman ve değer.
Yanlış: Stereo seste kanal sayısıyla çarpmayı unutmak. → **Doğrusu:** × 2. → **Neden:** Her an için sol ve sağ kanal ayrı ayrı örneklenir.
Yanlış: Video hesabında fps'i saniyedeki **bit** sanmak. → **Doğrusu:** fps saniyedeki **kare** sayısıdır; bir karenin bit sayısıyla çarpılır.

⚡ HIZLI TEKRAR

1. Video: zaman sırasıyla gösterilen kareler + ses + zamanlama + üst veri.
2. Veri hızı = G × Y × bpp × fps; Full HD 24 bit 30 fps ≈ 178 MiB/s, dakikada ≈ 10.4 GiB.
3. Ses: örnekleme (zaman) → nicemleme (değer, 2^b seviye) → kodlama (bitler).
4. Ses boyutu = hız × bit derinliği × kanal × süre.
5. Daha çok örnek/kare/bit = daha kaliteli ama daha büyük.

Ezberle: üç adımın adları ve sırası. **Türetebil:** her iki boyut formülü (görüntü formülünden).
Bu bölümden sonra şunları yapabiliyor olmalısın: video ve ses boyutunu hesaplamak; örnekleme ile nicemlemeyi ayırt etmek.

Bölüm 5 cevapları

Durup düşün 1. Hızlı değişimler iki örnek arasında kaybolur; ses boğuk, bozuk ya da tamamen farklı duyulur. Yüksek frekanslı sesler (zil, ince sesler) hiç yakalanamaz.

Yarı çözümlü. 1. 64,000 bit/s; 2. 3,840,000 bit; 3. 480,000 bayt; 4. 480,000 ÷ 1024 ≈ **468.75 KiB** (≈ 0.46 MiB).

Kendin çöz. (a) 2^4 = **16** seviye. (b) 48,000 × 24 × 2 × 120 = 276,480,000 bit ÷ 8 = **34,560,000 bayt** (≈ 32.96 MiB). (c) **İki katına** çıkar (veri hızı fps ile doğru orantılı).

BÖLÜM 6

Bellek Adresleme

◆ **BU BÖLÜMDE** Okuma süresi: 45 dk Zorluk: ●●●

- Bellek hücresi ve adres kavramlarını; adreslerin 0'dan başlayan ardışık sayılar olmasını
- Bayt adreslemeli belleği ve çok baytlı değişkenlerin ardışık adreslere yerleşmesini
- n adres biti → 2^n adreslenebilir bayt kuralını ve gerekli bit sayısını $\lceil \log_2 N \rceil$ ile bulmayı
- D1 s. 64'teki kapasite tablosunun her satırını kendin türetmeyi
- Adreslerin neden onaltılık ve 0x önekiyle yazıldığını

SINAV ÖNCELİĞİ: YÜKSEK ❖ Kaynak: D1 s. 60–65 / slayt 67–71

Sınav önceliği yüksek: öğrenme çıktısı "Relate memory capacity to the number of address bits", haftanın 5. sorusu "How many bits does a memory address require?" ve D1 s. 64 doğrudan soru üretmeye elverişli bir tablo. $\lceil \log_2 \rceil$ adımı en çok hata yapılan yer.

→ BU KONUYA NEDEN GEÇİYORUZ?

Şimdiye kadar sayıların, harflerin, piksellerin ve ses örneklerinin bitlere nasıl dönüştüğünü gördük. Ama bütün bu bitler **nerede** duruyor ve işlemci aradığını milyarlarca bit arasında nasıl buluyor? Cevap: belleğin her bölmesinin bir **adresi** var. Bu bölüm, Bölüm 1'deki 2^n kuralını son kez ve en önemli hâliyle kullanıyor: n bitlik adresle kaç bölmeye ulaşılabilir?

6.1 Bellek: numaralı çekmeceler

❖ Kaynak: D1 s. 61 / slayt 67

D1 s. 61'deki şekil belleği yan yana dizilmiş hücreler (**cell 0, cell 1, ...**) olarak gösteriyor; her hücrede 8 bitlik bir desen var (10111010, 10001101...).

Benzetme: okulun koridorundaki numaralı dolaplar. Dolap numarası **adres**, içindeki eşya **veri**. "17 numaralı dolabı aç" demek için içindekini bilmen gerekmez. **Benzetmenin bozulduğu yer:** Gerçek bellekte "boş dolap" yoktur; her hücre her an 0'lardan ve 1'lerden oluşan **bir değer** taşır. Bu yüzden programlamada ilk değeri verilmemiş bir değişkenden okunan değer "çöp" (rastgele görünen) bir değerdir.

∑ TANIM / FORMÜL — Adres (address)

- Bir adres, ana bellekteki bir konumu **benzersiz** olarak tanımlar.
- Adresler genellikle **0'dan başlayan ardışık** bir dizi oluşturur.
- İşlemci, belirli bir konumdaki veriyi **okumak ya da yazmak** için adresi kullanır.

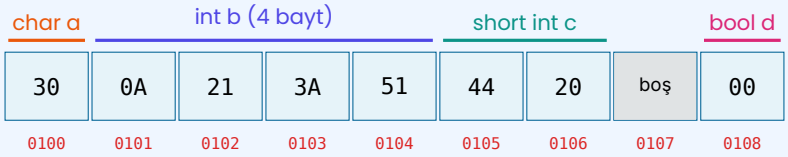
6.2 Bayt adreslemeli bellek ve değişkenler

❖ Kaynak: D1 s. 62 / slayt 68

∑ TANIM / FORMÜL — Bayt adreslemeli bellek (byte-addressable memory)

- Yaygın sistemlerde **her adres bir baytı** gösterir.
- Çok baytlı değerler **ardışık adresleri** kaplar.
- **32 bitlik bir tam sayı 4 bayt ve 4 adres** kaplar.
- Bir değişkenin adresi, kapladığı **ilk baytın (starting byte)** adresidir.

▶ ÖRNEK 6.1 — Slayt örneği [D1 s. 62 / slayt 68]: Dört değişken bellekte



Değişken	Tür	Bayt	Kapladığı adresler	Değişkenin adresi	İçerik (hex)
a	char	1	0100	**0100**	30 = 48 = **'0' karakteri**
b	int	4	0101–0104	**0101**	0A 21 3A 51
c	short int	2	0105–0106	**0105**	44 20
d	bool	1	0108	**0108**	00

Okuma kuralları: Slayttaki notlar: *"Each byte has an address. Each variable has the starting-byte address."* ve *"All values written in hexadecimal but binary in reality."* Adresler de değerler de onaltılık yazılmış ama bellekte aslında ikilik duruyor.
Dikkat çekici ayrıntı: 0107 boş bırakılmış. Slayt açıklamıyor (bkz. ek bilgi kutusu: hizalama).
En sık hata: b'nin adresini 0104 (son bayt) sanmak. Değişkenin adresi **ilk** baytının adresidir.

+ EK BİLGİ (KAYNAK DIŞI) — Hizalama (alignment) ve bayt sırası (endianness)

Slaytta yok, iki merak edilecek nokta:
Neden 0107 boş? Birçok derleyici değişkenleri işlemcinin daha hızlı okuyabileceği adreslere hizalar ve araya dolgu (padding) baytları koyabilir. Gerçek yerleşim derleyiciye ve sisteme bağlıdır; slayttaki şekil temsili bir örnektir.
b = 0A 21 3A 51 hangi sayı? Çok baytlı bir sayının baytlarının bellekte hangi sırayla durduğuna **bayt sırası (endianness)** denir. "Büyük uçlu" (big-endian) sistemde en anlamlı bayt önce gelir: 0x0A213A51. Bugün yaygın olan Intel/AMD ve ARM işlemciler "küçük uçlu" (little-endian) çalışır: aynı bellek içeriği 0x513A210A sayısı olarak okunur. Bu, "kural anlamı belirler" fikrinin bir örneği daha.

6.2.1 Kendin gör: C ile adresleri yazdırmak

Slaytlardaki değişken türleri (char, int, short) C/C++ türleri. Aşağıdaki küçük program, türlerin kaç bayt olduğunu ve bir `int` dizisinin elemanlarının adreslerini yazdırıyor (kaynak dışı ek örnek, D1 s. 62

ve s. 65'i doğrulamak için):

C

```c
#include <stdio.h>

int main(void) {
    char a = '0';          /* D1 s. 62'deki gibi: '0' karakteri */
    int  j[5];             /* D1 s. 65'teki gibi: 5 elemanlı int dizisi */

    printf("sizeof(char)  = %zu bayt\n", sizeof(char));
    printf("sizeof(short) = %zu bayt\n", sizeof(short));
    printf("sizeof(int)   = %zu bayt\n", sizeof(int));
    printf("a = '%c' = %d = 0x%X\n", a, a, a);

    for (int i = 0; i < 5; i++)
        printf("&j[%d] = %p\n", i, (void *)&j[i]);

    printf("j[1] - j[0] adres farki: %td bayt\n",
           (char *)&j[1] - (char *)&j[0]);
    return 0;
}
```

▶ Çıktı

```
sizeof(char)  = 1 bayt
sizeof(short) = 2 bayt
sizeof(int)   = 4 bayt
a = '0' = 48 = 0x30
&j[0] = 0x7ffd2d5d88f0
&j[1] = 0x7ffd2d5d88f4
&j[2] = 0x7ffd2d5d88f8
&j[3] = 0x7ffd2d5d88fc
&j[4] = 0x7ffd2d5d8900
j[1] - j[0] adres farki: 4 bayt
```

✔ gcc -Wall -Wextra ile uyarısız derlenip çalıştırıldı (64 bit Linux)

Çıktıyı okuma: `'0'` karakterinin değeri 48 = 0x30, tam slayttaki gibi. Dizi elemanlarının adresleri **4'er** artıyor (...f0, ...f4, ...f8, ...fc, ...900), çünkü her `int` 4 bayt. Onaltılıkta fc + 4 = 100 olduğuna dikkat: elde var. **Adreslerin kendisi her çalıştırmada değişir** (işletim sistemi programı her seferinde farklı yere yerleştirir); değişmeyen, aradaki 4 baytlık fark. `int`'in 4 bayt olması da sisteme bağlıdır; yaygın sistemlerde 4'tür.

✎ **DURUP DÜŞÜN**

1. Programdaki `%p` satırlarında adresler 12 onaltılık rakamla yazılmış (baştaki sıfırlar olmadan). D1 s. 65'teki eski Windows ekranında ise 8 rakam var (0012FF44). Bu fark neyi gösteriyor olabilir?

6.3 n adres biti → 2^n bayt

❖ Kaynak: D1 s. 63 / slayt 69

Adresler de bit desenleri. İşlemcinin adres için n bit kullandığını düşün. Bölüm 1'den biliyoruz: n bitle 2^n farklı desen var. Her desen bir adres, her adres bir bayt (bayt adreslemeli bellek) → **2^n bayta** ulaşılabilir.

∑ TANIM / FORMÜL — Adres biti ve kapasite

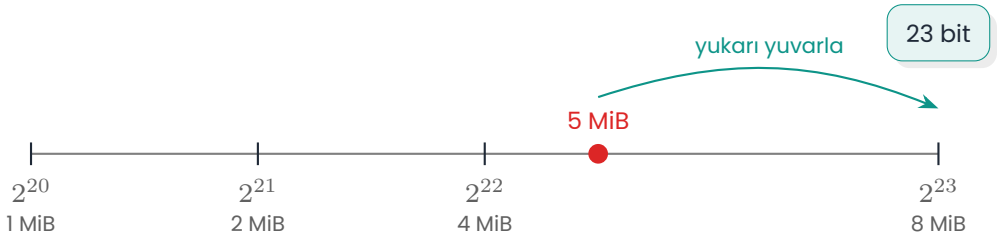
n adres biti → 2^n adreslenebilir bayt; adresler **0 ile $2^n - 1$** arasında.

Adres biti	10	20	30	32	40	42
Kapasite	2^{10} = 1 KiB	2^{20} = 1 MiB	2^{30} = 1 GiB	2^{32} = 4 GiB	2^{40} = 1 TiB	2^{42} = 4 TiB

Ters yön (gerekli bit sayısı): N bayt adreslenecekse **$n = \lceil \log_2 N \rceil$** (N'nin 2 tabanında logaritmasının **tavanı**). *"The required bit count is the ceiling of the base-2 logarithm of the number of addressable bytes."*

↺ HATIRLATMA — Logaritma ve tavan fonksiyonu

$\log_2 N$ sorusu: "2'yi kaçıncı kuvvete yükseltirsem N olur?" $\log_2 8 = 3$ (çünkü $2^3 = 8$), $\log_2 1024 = 10$.
N, 2'nin tam kuvveti değilse logaritma tam sayı çıkmaz: $\log_2 5{,}000{,}000 \approx 22.25$.
Tavan (ceiling), $\lceil x \rceil$: x'ten büyük veya ona eşit en küçük tam sayı; yani **yukarı yuvarla**. $\lceil 22.25 \rceil = 23$, $\lceil 3 \rceil = 3$, $\lceil 3.01 \rceil = 4$.
Neden yukarı? 22 bit yalnızca 2^{22} = 4,194,304 adres verir, 5 milyon bayta yetmez. Eksik kalmak olmaz, fazladan adres sorun değil; o yüzden her zaman yukarı yuvarlarız. Bölüm 1'deki "30 öğrenci → 5 bit" örneğinin aynısı.
Hesap makinesi olmadan: log hesaplamana gerek yok. N'yi 2'nin kuvvetleri arasına sıkıştır: $2^k < N \le 2^{k+1}$ ise cevap $k + 1$. (N tam olarak 2^k ise cevap k.)



Şekil 6.1: 5 MiB, 2^{22} (4 MiB) ile 2^{23} (8 MiB) arasında kalır. 22 bit yetmez, 23 bit gerekir. (Eksen logaritmik değil, sadece sıralamayı gösteriyor.)

6.3.1 D1 s. 64'teki tablo, satır satır

❖ Kaynak: D1 s. 64 / slayt 70

▶ ÖRNEK 6.2 — Slayt örneği [D1 s. 64 / slayt 70]: Kapasiteden adres bitine

Yöntem (her satır için): (1) Kapasiteyi bayt cinsinden 2'nin kuvvetiyle yaz (KiB = 2^{10}, MiB = 2^{20}, GiB = 2^{30}, TiB = 2^{40}). (2) Tam kuvvetse üs cevaptır. (3) Değilse iki kuvvet arasına sıkıştır, büyük olanın üssünü al.

Kapasite	Bayt sayısı	Arasında	Gerekli bit
1 KiB	2^{10}	tam kuvvet	**10**
5 MiB	$5 \times 2^{20} = 5{,}242{,}880$	$2^{22} = 4 \times 2^{20} < 5 \times 2^{20} < 8 \times 2^{20} = 2^{23}$	**23**
128 MiB	$2^7 \times 2^{20} = 2^{27}$	tam kuvvet	**27**
4 GiB	$2^2 \times 2^{30} = 2^{32}$	tam kuvvet	**32**
3 TiB	3×2^{40}	$2^{41} = 2 \times 2^{40} < 3 \times 2^{40} < 4 \times 2^{40} = 2^{42}$	**42**

Püf noktası: Katsayıyı 2'nin kuvvetine çevir ve üsleri topla: $128 = 2^7 \rightarrow 2^7 \cdot 2^{20} = 2^{27}$. Katsayı 2'nin kuvveti değilse (5, 3, 12...) bir üst kuvvete yuvarla: $5 \rightarrow 8 = 2^3$, yani $2^3 \cdot 2^{20} = 2^{23}$.
En sık hata: 5 MiB için "$5 \times 20 = 100$ bit" ya da "$\log_2 5 + 20 = 22.3 \rightarrow 22$" demek. Aşağı yuvarlamak 5 MiB'ın bir kısmını adressiz bırakır.

▶ ÖRNEK 6.3 — Ek örnek: 12 KiB ve 64 KiB

12 KiB: 12×2^{10}. 12, 8 (2^3) ile 16 (2^4) arasında → $2^{13} < 12$ KiB $< 2^{14}$ → **14 bit.** Kontrol: $2^{13} = 8{,}192 < 12{,}288 \leq 16{,}384 = 2^{14}$ ✓.
Ters yön, 16 adres biti: $2^{16} = 2^6 \cdot 2^{10}$ = **64 KiB**; adresler 0x0000 ile 0xFFFF arası (16 bit = 4 onaltılık rakam).

▶ ÖRNEK 6.4 — Yarı çözümlü: 768 MiB ve 2 GiB

768 MiB = 768×2^{20}. $512 = 2^9 < 768 < 1024 = 2^{10} \rightarrow 2^{29} <$ 768 MiB $< 2^{30} \rightarrow$ ... bit.
2 GiB = $2^1 \times 2^{30} = 2^{\wedge}... \rightarrow$ ... bit.

6.4 Adresler neden onaltılık yazılır?

❖ Kaynak: D1 s. 65 / slayt 71

32 bitlik bir adresi ikilik yazmak 32 karakter demek: 00000000000100101111111101000100. Aynı adres onaltılık: **0012FF44**. Bölüm 2'deki kural: **her onaltılık rakam 4 adres biti**; 32 bit = 8 rakam.

D1 s. 65'teki ekran görüntüsünde bir programın dizi adresleri var: j[0] = 0012FF44, j[1] = 0012FF48, j[2] = 0012FF4C, ... Fark her seferinde 4 (44 → 48 → 4C → 50 → 54), çünkü her eleman 4 baytlık bir `int`. 48 + 4 = 4C (12) ve 4C + 4 = 50: onaltılıkta elde (C + 4 = 16 = 0x10).

∑ TANIM / FORMÜL — Onaltılık adres gösterimi

- Onaltılık, uzun ikilik adresler için **kısa bir biçim** sağlar.
- Her onaltılık rakam **dört adres bitini** temsil eder.
- Hata ayıklayıcılar (debugger) ve bellek dökümleri (memory dump) adresleri genellikle **0x** önekiyle yazar.

✎ KENDİN ÇÖZ

(a) 256 MiB'lık bir belleği adreslemek için kaç bit gerekir? (b) 20 KiB için? (c) 24 adres biti kaç baytlık belleği adresleyebilir, MiB olarak yaz. En büyük adres onaltılık olarak nedir? (d) Bir `short` dizisinin (her eleman 2 bayt) ilk elemanının adresi 0x1FFE ise üçüncü elemanın (indeks 2) adresi nedir?

Bağlantılar

- Bölüm 1'deki **2^n** ve "N nesne için en az kaç bit" sorusunun tam karşılığı: $n = \lceil \log_2 N \rceil$.
- Bölüm 2'deki **onaltılık** ve **0x**: adresler, 4 bit = 1 rakam.
- Bölüm 3'teki karakter kodları ve Bölüm 4'teki pikseller bellekte bu ardışık adreslere bayt bayt yerleşir; görüntünün "doğrusal saklanması" tam olarak budur.

⚠ SINAVDA DİKKAT

- Bayt adreslemeli: **1 adres = 1 bayt**; değişkenin adresi = **ilk** baytının adresi.
- int (32 bit) = 4 bayt = 4 adres; ardışık int adresleri **4'er** artar.
- n bit → **2^n** bayt; adres aralığı 0 … $2^n - 1$.
- Gerekli bit = **$\lceil \log_2 N \rceil$** → her zaman **yukarı** yuvarla.
- Kuvvetler: KiB 2^{10}, MiB 2^{20}, GiB 2^{30}, TiB 2^{40}. Katsayıyı 2'nin kuvvetine çevirip üsleri topla.

❖ BU KONUDAN NASIL SORU GELİR?

1. Hesap (çok olası): *"How many address bits are needed to address 6 GiB of byte-addressable memory?"* İpucu: 6×2^{30}; $4 < 6 < 8 \rightarrow 2^{32} < \ldots < 2^{33} \rightarrow 33$.
2. Ters hesap: *"A processor uses 36-bit addresses. What is the maximum memory size?"* İpucu: $2^{36} = 2^6 \cdot 2^{30} = 64$ GiB.
3. Bellek yerleşimi: "Bir int değişken 0x0200'de başlıyorsa hangi adresleri kaplar? Bir sonraki int nerede başlar?" İpucu: 0x0200–0x0203; 0x0204.
4. Kavramsal: *"Why are memory addresses usually written in hexadecimal?"* İpucu: 4 bit = 1 rakam, kısa ve ikiliğe kolay çevrilir.

✘ SIK YAPILAN HATA

Yanlış: $\lceil \log_2 \rceil$ yerine aşağı yuvarlamak (5 MiB → 22 bit). → **Doğrusu:** 23. → **Neden:** 22 bit sadece 4 MiB'ı adresler; 1 MiB'lık kısım ulaşılamaz kalır.
Yanlış: "1 KiB = 1000 bayt, o hâlde $\lceil \log_2 1000 \rceil = 10$." → Sonuç tesadüfen aynı ama gerekçe yanlış. **Doğrusu:** 1 KiB = 1024 = 2^{10} bayt, tam kuvvet. → **Neden:** Slaytlar ikili önekleri kullanıyor; 1000'li hesap başka sorularda (ör. 1 MB = 10^6) farklı sonuç verir.
Yanlış: Adres bitlerini veri bitleriyle karıştırmak: "32 bitlik bir sistem 32 bayt bellek adresler." → **Doğrusu:** 2^{32} bayt = 4 GiB. → **Neden:** n bit 2^n farklı adres üretir; n'nin kendisi bayt sayısı değildir.

⚡ **HIZLI TEKRAR**

1. Bellek ardışık hücrelerden oluşur; adres 0'dan başlar, her konumu benzersiz tanımlar.
2. Bayt adreslemede her adres bir bayt; çok baytlı değişkenler ardışık adresleri kaplar, adresleri ilk baytlarıdır.
3. n adres biti → 2^n bayt (0 … $2^n - 1$).
4. N bayt için n = $\lceil \log_2 N \rceil$: 2'nin kuvvetleri arasına sıkıştır, yukarı yuvarla.
5. Adresler onaltılık yazılır: 1 rakam = 4 bit, 0x öneki.

Ezberle: 2^{10} = KiB, 2^{20} = MiB, 2^{30} = GiB, 2^{40} = TiB; 32 bit → 4 GiB. **Türetebil:** D1 s. 64'teki her satır.
Bu bölümden sonra şunları yapabiliyor olmalısın: kapasiteden adres bitini ve adres bitinden kapasiteyi bulmak; bir değişkenin kapladığı adresleri yazmak; onaltılık adres farklarını yorumlamak.

Bölüm 6 cevapları

Durup düşün 1. D1 s. 65'teki ekran 32 bitlik bir sisteme ait (8 hex rakam = 32 bit). Bu kitaptaki program 64 bitlik bir sistemde çalıştı; adresler 64 bit olarak tutulur, `%p` baştaki sıfırları yazmadığı için 12 rakam göründü (tam yazılsa 16 rakam = 64 bit olurdu).

Yarı çözümlü. 768 MiB → **30 bit**. 2 GiB = 2^{31} → **31 bit**.

Kendin çöz. (a) 256 MiB = $2^8 \cdot 2^{20} = 2^{28}$ → **28 bit**. (b) 20 KiB: 16 < 20 < 32 → 2^{14} < 20 KiB < 2^{15} → **15 bit**. (c) $2^{24} = 2^4 \cdot 2^{20}$ = **16 MiB** (16,777,216 bayt); en büyük adres $2^{24} - 1$ = **0xFFFFFF** (24 bit = 6 hex rakam). (d) Her eleman 2 bayt: indeks 2'nin adresi 0x1FFE + 2 · 2 = 0x1FFE + 4 = **0x2002** (E + 4 = 18 = 0x12 → elde).

Konular Arası Bağlantılar

Bu haftanın bütün konuları üç fikirden doğuyor: **(1) n bit → 2^n desen**, **(2) bitlerin anlamını kodlama kuralı belirler**, **(3) onaltılık sistem bitleri kısaltır.** Aşağıdaki tablo aynı formülün haftanın her yerinde nasıl tekrar ettiğini gösteriyor.

2^n kuralının kılığı	Bit	Ne sayılıyor	Örnek	Bölüm
n bit → 2^n değer	n	farklı değer	8 bit → 256	1
4 bit = 1 hex rakam	4	onaltılık rakam	2^4 = 16	2
ASCII	7	karakter kodu	2^7 = 128	3
8 bit kod sayfası	8	karakter kodu	2^8 = 256	3
Renk derinliği	b bpp	renk	24 → 16,777,216	4
Palet indeksi	8	palet rengi	256	4
Nicemleme	b bit/örnek	seviye	4 bit → 16	5
Adres biti	n	adreslenebilir bayt	32 → 4 GiB	6

Ters yön hep aynı soru: **N tane şeyi ayırt etmek için en az kaç bit?** → n = ⌈$\log_2$ N⌉ (30 öğrenci → 5 bit; 5 MiB → 23 bit).

Boyut formülleri (sıkıştırılmamış)	
Görüntü	G × Y × bpp (bit)
Video	G × Y × bpp × fps × süre (bit)
Ses	örnekleme hızı × bit derinliği × kanal × süre (bit)
Metin	karakter sayısı × karakter başına bayt (ASCII 1; UTF-8 1–4; UTF-16 2/4; UTF-32 4)

"Kural anlamı belirler" örnekleri	Aynı bitler, farklı anlam
01000001	sayı 65 / ASCII 'A' / koyu gri
bayt 0x9F	CP857'de ş / CP437'de ƒ
C5 9F	UTF-8'de ş / Windows-1252'de "ÅŸ"
1-bit piksel	0 = siyah mı beyaz mı? Dosya biçimine bağlı (D1 s. 44–45)
0A 21 3A 51	big-endian 0x0A213A51 / little-endian 0x513A210A

Onaltılığın göründüğü yerler	Örnek
Karakter kodları	A = 0x41, U+00E7
Renkler	#FF8000

Onaltılığın göründüğü yerler	Örnek
Bellek adresleri	0x0012FF44
Hata kodları	STOP 0x00000050

Sınav Öncesi Hızlı Tekrar

Temel tanımlar

- **Veri:** ham değerler (gözlem, ölçüm, sembol). **Bilgi:** bir soruyu cevaplamak için yorumlanmış veri. Fark: **bağlam + yorum**.
- **Kodlama:** veriyi bit dizisine çevirmek. **Kod çözme:** bit dizisini kullanıcı biçimine geri çevirmek.
- **Bit:** 0/1; **bayt:** 8 bit; **bit deseni** uzunluğu = bit sayısı. Bitlerin anlamını **kodlama kuralı** belirler.
- **Konumsal sistem:** değer = rakam × taban^konum; rakam < taban; tabanı alt indis veya önekle yaz.
- **ASCII:** 7 bit, 128 kod. **Kod sayfası:** 8 bit, 128–255 değişir. **EBCDIC:** IBM mainframe, 8 bit, ASCII ile uyumsuz.
- **Unicode:** karakter kimliği (kod noktası, U+XXXX). **UTF-8/16/32:** bayt temsili.
- **Bitmap:** piksel matrisi. **Vektör:** geometrik nesneler; ölçeklenince keskin.
- **Örnekleme:** zamanda ölçmek. **Nicemleme:** değeri sonlu seviyeye eşlemek.
- **Adres:** bellek konumunu benzersiz tanımlar; bayt adreslemede 1 adres = 1 bayt; değişkenin adresi = ilk baytı.

Formül listesi

Ne	Formül
n bitle değer sayısı	2^n (en büyük: $2^n - 1$)
N şey için gereken bit	$\lceil \log_2 N \rceil$
Taban gösterimi	$(d_k \ldots d_0)_b = \Sigma\, d_i \cdot b^i$
Renk sayısı	2^b (b = bpp)
Görüntü boyutu (bit)	G × Y × bpp
Video veri hızı (bit/s)	G × Y × bpp × fps
Ses boyutu (bit)	hız × bit derinliği × kanal × süre
Adreslenebilir bayt	2^n (n adres biti)
Birimler	bit ÷ 8 = bayt; KiB 2^{10}, MiB 2^{20}, GiB 2^{30}, TiB 2^{40}

Ezber tabloları

2^0	2^1	2^2	2^3	2^4	2^5	2^6	2^7	2^8
1	2	4	8	16	32	64	128	256

2^9	2^{10} (KiB)	2^{16}	2^{20} (MiB)	2^{24}	2^{30} (GiB)
512	1,024	65,536	1,048,576	16,777,216	1,073,741,824

2^{32} = 4 GiB = 4,294,967,296 bayt.

Hex	0–9	A	B	C	D	E	F
Onluk	0–9	10	11	12	13	14	15
İkilik	0000–1001	1010	1011	1100	1101	1110	1111

ASCII çapaları: boşluk 32, '0' 48 (0x30), A 65 (0x41), a 97 (0x61); büyük → küçük = +32.

UTF karakter başına: UTF-8 1–4 (ASCII 1, Türkçe harfler 2), UTF-16 2/4, UTF-32 4.

Dönüşüm yöntemleri

Yön	Yöntem
2 → 10, 16 → 10	ağırlıkları topla
10 → 2, 10 → 16	tabana böl, kalanları **aşağıdan yukarı** oku; 10–15 → A–F
2 → 16	**sağdan** 4'erli grupla, başa 0 ekle
16 → 2	her rakam **tam 4 bit**

Kritik ayrımlar

- Kaç **değer** (2^n) ≠ en büyük **değer** ($2^n - 1$).
- **Çözünürlük** (kaç piksel) ≠ **renk derinliği** (piksel başına bit).
- **Örnekleme** (zaman) ≠ **nicemleme** (değer).
- **Kod noktası** (kimlik) ≠ **kodlama** (bayt) ≠ **font** (çizim).
- '7' karakteri (55 = 0x37) ≠ 7 sayısı.
- MiB (2^{20}) ≠ MB (10^6).

En sık 8 hata

1. Kalanları yukarıdan aşağı okumak.
2. Soldan 4'erli gruplamak.
3. En sağ basamağın ağırlığını 1 yerine 2 almak.
4. Bitleri bayt sanmak (8'e bölmeyi unutmak).
5. 2^n yerine n^2 ya da 2n yazmak.
6. $\lceil \log_2 \rceil$'ü aşağı yuvarlamak.
7. UTF-8'de her karakteri 1 bayt saymak.
8. Stereo seste × 2'yi, videoda × süreyi unutmak.

Sınav sabahı kontrol listesi

☐ Kâğıdın köşesine 0–F ikilik tablosunu ve 2^0…2^{10}'u yaz.

- ☐ Her dönüşümde tabanı ($_2$, $_{16}$, 0x) yaz ve ters yönden kontrol et.
- ☐ Boyut hesaplarında her satıra birim yaz: piksel → bit → bayt → MiB.
- ☐ “Kaç bit gerekir” sorularında yukarı yuvarla.
- ☐ Tanım sorularında anahtar kelimeleri kullan: raw / interpreted, code point / encoding, sampling / quantization.

Deneme Sınavı

Süre: 60 dakika. **Bölüm A:** 12 çoktan seçmeli soru (her biri 3 puan, toplam 36). **Bölüm B:** 8 klasik soru (toplam 64). Hesap makinesi kullanmadan çöz; bütün ara adımları yaz. Sorular İngilizce, altında Türkçe çevirisi var. Cevap anahtarı bir sonraki bölümde: önce kendin çöz!

Bölüm A: Çoktan seçmeli

A1. *(Kolay)* Which of the following is an example of **information** rather than data? *Aşağıdakilerden hangisi veri değil, bilgi örneğidir?*

A) 62, 71, 84, 95 B) The class average is 78 C) 35209852715 D) 0x2A

❖ Kaynak: D1 s. 4–5

A2. *(Kolay)* How many different values can be represented with 10 bits? *10 bit ile kaç farklı değer gösterilebilir?*

A) 10 B) 100 C) 512 D) 1024

❖ Kaynak: D1 s. 10

A3. *(Kolay)* What is the decimal value of 0x7D? *0x7D'nin onluk değeri nedir?*

A) 125 B) 113 C) 127 D) 213

❖ Kaynak: D1 s. 19, 25

A4. *(Orta)* What is 101101_2 in hexadecimal? *101101_2 onaltılık sistemde nedir?*

A) B1 B) 55 C) 2D D) 45

❖ Kaynak: D1 s. 22, 27

A5. *(Orta)* Which of the following is **not** a valid number in the given base? *Aşağıdakilerden hangisi belirtilen tabanda geçerli bir sayı **değildir**?*

A) 1101_2 B) 1081_8 C) $7F_{16}$ D) 999_{10}

❖ Kaynak: D1 s. 14, 17

A6. *(Kolay)* Which statement about ASCII is correct? *ASCII ile ilgili hangi ifade doğrudur?*

A) It uses 7 bits and defines 128 codes. B) It uses 8 bits and defines 256 codes. C) It uses 16 bits and defines 65,536 codes. D) It uses 7 bits and defines 256 codes.

❖ Kaynak: D1 s. 32

A7. *(Orta)* In ASCII, the code of 'G' is 71. What is the code of 'g'? *ASCII'de 'G' harfinin kodu 71'dir. 'g' harfinin kodu nedir?*

A) 72 B) 39 C) 70 D) 103

❖ Kaynak: D1 s. 32–33

A8. *(Orta)* Which statement is correct? *Hangi ifade doğrudur?*

A) Unicode stores every character in exactly 2 bytes.
B) UTF-8 is not compatible with ASCII.
C) A code point identifies a character; UTF-8 defines how it is stored as bytes.
D) UTF-32 uses 1 to 4 bytes per character.

❖ Kaynak: D1 s. 39–41, 66

A9. *(Orta)* How many different colors can a pixel show with a color depth of 6 bits per pixel? *Piksel başına 6 bit renk derinliğiyle bir piksel kaç farklı renk gösterebilir?*

A) 64 B) 36 C) 12 D) 6

❖ Kaynak: D1 s. 47

A10. *(Zor)* How many bytes does the word "Ağaç" occupy in UTF-8? *"Ağaç" kelimesi UTF-8 ile kaç bayt yer kaplar?*

A) 4 B) 5 C) 7 D) 6

❖ Kaynak: D1 s. 40–41 (UTF-8 1–4 bayt); karakter başına bayt sayısı kaynak dışı genişletmeden (Bölüm 3 ek bilgi)

A11. *(Orta)* In analog-to-digital audio conversion, mapping each measured value to one of a finite number of levels is called: *Analog sesi sayısala çevirirken, ölçülen her değerin sonlu sayıdaki seviyeden birine eşlenmesine ne denir?*

A) sampling B) quantization C) decoding D) indexing

❖ Kaynak: D1 s. 59

A12. *(Zor)* How many address bits are required to address 3 MiB of byte-addressable memory? *Bayt adreslemeli 3 MiB'lık bir belleği adreslemek için kaç adres biti gerekir?*

A) 21 B) 23 C) 22 D) 3

❖ Kaynak: D1 s. 63–64

Bölüm B: Klasik sorular

B1. *(Kolay, 6 puan)* Convert 89_{10} to binary and to hexadecimal. Show your work. *89_{10} sayısını ikilik ve onaltılık sisteme çevir. Çözümünü göster.*

❖ Kaynak: D1 s. 24–25

B2. *(Orta, 6 puan)* Convert 0xB6 to binary and to decimal. Show your work. *0xB6 sayısını ikilik ve onluk sisteme çevir. Çözümünü göster.*

❖ Kaynak: D1 s. 18, 27

B3. *(Orta, 8 puan)* (a) Why do computers use the binary system? (b) Why is hexadecimal widely used by programmers? Give two places where hexadecimal is used. *(a) Bilgisayarlar neden ikilik sistemi kullanır? (b) Onaltılık sistem programcılar tarafından neden yaygın kullanılır? Onaltılığın kullanıldığı iki yer yaz.*

❖ Kaynak: D1 s. 16, 20–21

B4. *(Orta, 8 puan)* An image is 1024 × 600 pixels with 24-bit color. (a) Calculate its uncompressed size in bytes and MiB. (b) What would the size of the pixel data (excluding the palette table) be

(in bytes) if a 256-color palette were used instead? *Bir görüntü 1024 × 600 piksel ve 24 bit renkli. (a) Sıkıştırılmamış boyutunu bayt ve MiB olarak hesapla. (b) Bunun yerine 256 renklik bir palet kullanılsaydı piksel verisi (palet tablosu hariç) kaç bayt olurdu?*

❖ Kaynak: D1 s. 43, 52–53

B5. *(Zor, 10 puan)* (a) Calculate the uncompressed data rate of a video with 1280 × 720 resolution, 24-bit color and 30 fps in bits per second, bytes per second and MiB/s. (b) How much storage (in GiB, rounded to two decimals) does 2 minutes of this video need? (c) How many bytes does 30 seconds of uncompressed audio need at 44,100 Hz, 16-bit, stereo? *(a) 1280 × 720 çözünürlüklü, 24 bit renkli, 30 fps bir videonun sıkıştırılmamış veri hızını bit/s, bayt/s ve MiB/s olarak hesapla. (b) Bu videonun 2 dakikası kaç GiB yer tutar (iki ondalığa yuvarla)? (c) 44,100 Hz, 16 bit, stereo, 30 saniyelik sıkıştırılmamış ses kaç bayttır?*

❖ Kaynak: D1 s. 58–59; (c) ses boyutu formülü kaynak dışı genişletme (Bölüm 5)

B6. *(Orta, 8 puan)* The Unicode code point of the Turkish letter ‘ş’ is U+015F and its UTF-8 bytes are C5 9F. (a) Explain the difference between a code point and an encoding using this example. (b) A Turkish text file shows “ÅŸ” instead of “ş”. Explain why. *Türkçe ‘ş’ harfinin Unicode kod noktası U+015F, UTF-8 baytları C5 9F’dir. (a) Bu örnek üzerinden kod noktası ile kodlama arasındaki farkı açıkla. (b) Türkçe bir metin dosyasında “ş” yerine “ÅŸ” görünüyor. Nedenini açıkla.*

❖ Kaynak: D1 s. 34, 39, 66

B7. *(Zor, 10 puan)* (a) How many address bits are needed for 6 GiB of byte-addressable memory? (b) A system uses 20-bit addresses. How much memory can it address? Write the largest address in hexadecimal. (c) A 32-bit integer is stored starting at address 0x00FC. Which addresses does it occupy, and where can the next integer start? *(a) Bayt adreslemeli 6 GiB bellek için kaç adres biti gerekir? (b) Bir sistem 20 bitlik adresler kullanıyor. Ne kadar belleği adresleyebilir? En büyük adresi onaltılık yaz. (c) 32 bitlik bir tam sayı 0x00FC adresinden başlayarak saklanıyor. Hangi adresleri kaplar, bir sonraki tam sayı nerede başlayabilir?*

❖ Kaynak: D1 s. 62–65

B8. *(Zor, 8 puan)* A web page uses the color #4B0082. (a) Write its R, G and B values in decimal. (b) How many bits per pixel does this color format use and how many different colors can it represent? (c) What is the uncompressed size of a 100 × 100 image in this format, in bytes? *Bir web sayfası #4B0082 rengini kullanıyor. (a) R, G ve B değerlerini onluk yaz. (b) Bu renk biçimi piksel başına kaç bit kullanır ve kaç farklı renk gösterebilir? (c) Bu biçimdeki 100 × 100 bir görüntünün sıkıştırılmamış boyutu kaç bayttır?*

❖ Kaynak: D1 s. 47–52

Ayrıntılı Cevap Anahtarı

Bütün sayısal cevaplar Python ile kontrol edildi; ayrıca sorular, cevap anahtarını görmeyen bağımsız bir çözücüye çözdürüldü ve cevaplar birebir uyuştu. Çözücünün işaret ettiği iki belirsizlik (A10'da bir şık, B5'te yuvarlama) sorularda giderildi.

Soru	A1	A2	A3	A4	A5	A6	A7	A8	A9	A10	A11	A12
Cevap	B	D	A	C	B	A	D	C	A	D	B	C

Bölüm A

A1 → B. "The class average is 78", ham notlar (62, 71, 84, 95) yorumlanarak "sınıfın ortalaması kaç?" sorusuna verilmiş cevaptır: bilgi. **A:** ham ölçümler, veri. **C:** bağlamı olmayan çıplak sayı, veri. **D:** sadece onaltılık yazılmış bir değer (42); bağlam yok, veri. *Yanılgı: "cümle olan bilgi, sayı olan veri".*

A2 → D. n bit → 2^n: 2^{10} = 1024. **A (10):** bit sayısının kendisi. **B (100):** 10^2 ile karıştırma (n^2 hatası). **C (512):** 2^9; bir eksik üs ya da "en büyük değer"i yarıya bölme karışıklığı. *En büyük değer 1023 olurdu; soru değer sayısını soruyor.*

A3 → A. 7 · 16 + D(13) = 112 + 13 = 125. **B (113):** D'yi 1 alma (7 · 16 + 1). **C (127):** 0x7F ile karıştırma. **D (213):** 7D'yi onluk sanıp 7 · 10 + … gibi anlamsız birleştirme / taban karışıklığı.

A4 → C. Sağdan 4'erli: 10 | 1101 → 0010 | 1101 → 2 D. **A (B1):** soldan gruplama hatası (1011 | 01). **B (55):** sekizlik karşılığı (101 | 101 = 5 5). **D (45):** onluk değeri; taban karışıklığı.

A5 → B. 1081_8 içinde **8** rakamı var; sekizlikte rakamlar 0–7. Diğerleri geçerli: 1101_2 (yalnız 0/1), $7F_{16}$ (F = 15 < 16), 999_{10}.

A6 → A. ASCII 7 bit, 2^7 = 128 kod (D1 s. 32). **B:** 8 bitlik kod sayfaları ("extended ASCII"). **C:** slayttaki ilk Unicode tanımı (D1 s. 37). **D:** bit sayısı ile kod sayısı uyuşmuyor (7 bit 256 değer veremez).

A7 → D. Küçük harf = büyük harf + 32: 71 + 32 = 103. **A (72):** bir sonraki harf 'H'. **B (39):** 32 çıkarmak (yön hatası). **C (70):** 'F'.

A8 → C. Kod noktası karakterin kimliği (ş = U+015F), UTF-8 bayt temsili (C5 9F). **A:** UTF-16'da bile 2 **veya 4** bayt; Unicode kendisi bayt sayısı belirlemez. **B:** UTF-8 ASCII uyumludur. **D:** UTF-32 sabit 4 bayttır; 1–4 bayt UTF-8'in özelliği.

A9 → A. 2^6 = 64. **B (36):** 6^2. **C (12):** 6 · 2. **D (6):** bit sayısı.

A10 → D. A (1) + ğ (2) + a (1) + ç (2) = 6 bayt. **A (4):** karakter sayısı; her karakteri 1 bayt sanmak. **B (5):** Türkçe harflerden yalnızca birini 2 bayt saymak. **C (7):** ğ'yi 3 bayt saymak (yalnızca U+0800 ve üstü 3 bayttır; ğ = U+011F).

A11 → B. Nicemleme = değeri sonlu seviyelere eşlemek. **A:** örnekleme, zamanda belirli anlarda ölçmek. **C:** bitleri kullanıcı biçimine geri çevirmek. **D:** palet indeksi (görüntü konusu).

A12 → C. 3 MiB = 3 × 2^{20}; $2 < 3 < 4 \rightarrow 2^{21} < 3$ MiB $< 2^{22} \rightarrow \lceil \log_2 \rceil = \mathbf{22}$. **A (21):** aşağı yuvarlama, 2 MiB'ı adresler, yetmez. **B (23):** 5 MiB örneğinin (D1 s. 64) cevabını ezberden yazmak. **D (3):** sadece katsayıya bakmak.

Bölüm B

B1 (6 puan). 2'ye bölme: 89→44 k1, 44→22 k0, 22→11 k0, 11→5 k1, 5→2 k1, 2→1 k0, 1→0 k1. Yukarı oku: **1011001_2**. *(3 puan: bölme tablosu + doğru okuma yönü.)* Onaltılık: 89 ÷ 16 = 5 k 9, 5 ÷ 16 = 0 k 5 → **0x59**; ya da 0101 | 1001 → 5 9. *(2 puan.)* Kontrol: 64 + 16 + 8 + 1 = 89; 5 · 16 + 9 = 89. *(1 puan: kontrol satırı.)*

B2 (6 puan). B = 1011, 6 = 0110 → **10110110_2** *(3 puan; her rakam tam 4 bit)*. Onluk: 11 · 16 + 6 = 176 + 6 = **182** *(3 puan)*. Çapraz kontrol: 128 + 32 + 16 + 4 + 2 = 182.

B3 (8 puan). (a) Donanım iki fiziksel durumu güvenilir biçimde ayırt edebilir: düşük/yüksek gerilim, transistörün kapalı/açık olması, manyetik yön ya da ışık seviyesinin iki aralığa eşlenmesi. İki seviye arasında geniş **gürültü payı** bırakılabildiği için parazit değeri bozmaz; çok seviyeli sistemde aralıklar daralır ve hata artar. *(4 puan: "iki durum + güvenilirlik/gürültü payı" ana fikri, en az bir fiziksel örnek.)* (b) $2^4 = 16$ olduğundan her onaltılık rakam tam 4 bite karşılık gelir; uzun bit dizileri 4 kat kısalır ve ikiliğe hesapsız çevrilir. Kullanım: bellek adresleri, hata kodları (0x00000050), makine kodu/hata ayıklama çıktıları, RGB renkleri (#RRGGBB), ağ protokolü alanları (herhangi ikisi). *(4 puan.)*

B4 (8 puan). (a) 1024 × 600 = 614,400 piksel; × 24 = 14,745,600 bit; ÷ 8 = **1,843,200 bayt**; ÷ 1,048,576 ≈ **1.76 MiB** (tam: 1.7578125). *(5 puan: her adım 1 puan + birim.)* (b) 256 = 2^8 renk → 8 bit = 1 bayt/piksel → **614,400 bayt** (paletin kendisi ayrıca 256 × 3 = 768 bayt). *(3 puan.)*

B5 (10 puan). (a) Kare: 1280 × 720 × 24 = 22,118,400 bit; × 30 = **663,552,000 bit/s**; ÷ 8 = **82,944,000 bayt/s**; ÷ 1,048,576 ≈ **79.10 MiB/s**. *(5 puan.)* (b) 120 s × 82,944,000 = 9,953,280,000 bayt; ÷ 2^{30} = 1,073,741,824 ≈ **9.27 GiB**. *(3 puan; 60 ile değil 120 ile çarpmak ve GiB'a doğru bölmek.)* (c) 44,100 × 16 × 2 × 30 = 42,336,000 bit ÷ 8 = **5,292,000 bayt** (≈ 5.05 MiB). *(2 puan; stereo × 2 unutulursa 1 puan.)*

B6 (8 puan). (a) **Kod noktası** U+015F, 'ş' karakterinin Unicode'daki benzersiz kimlik numarasıdır; hangi dosyada, hangi sistemde olursa olsun değişmez. **Kodlama** (UTF-8), bu numaranın dosyada hangi baytlarla saklanacağını belirleyen kuraldır: UTF-8'de C5 9F (2 bayt); UTF-16'da 01 5F, UTF-32'de 4 bayt olurdu. Aynı karakter, aynı kod noktası, farklı baytlar. *(4 puan.)* (b) Dosya UTF-8 ile yazılmış ama 8 bitlik bir kod sayfasıyla (ör. Windows-1252) okunmuş. Okuyan program iki baytlık C5 9F dizisini iki ayrı karakter sanmış: o kod sayfasında C5 = Å, 9F = Ÿ. Kodlama uyuşmazlığı → bozuk metin (D1 s. 34: yanlış kod sayfası bozuk metin üretir). *(4 puan.)*

B7 (10 puan). (a) 6 GiB = 6 × 2^{30}; $4 < 6 < 8 \rightarrow 2^{32} < 6$ GiB $< 2^{33} \rightarrow$ **33 bit**. *(3 puan.)* (b) 2^{20} bayt = **1 MiB** (1,048,576 bayt); adresler 0'dan $2^{20} - 1$'e: en büyük adres **0xFFFFF** (20 bit = 5 onaltılık rakam, hepsi F). *(4 puan.)* (c) 32 bit = 4 bayt → **0x00FC, 0x00FD, 0x00FE, 0x00FF**; sonraki tam sayı **0x0100**'de başlayabilir (FF + 1 = 100, elde). *(3 puan.)*

B8 (8 puan). (a) 4B = 4 · 16 + 11 = **75**, 00 = **0**, 82 = 8 · 16 + 2 = **130** → (75, 0, 130), koyu mor-lacivert bir renk. *(3 puan.)* (b) Her kanal 2 hex rakam = 8 bit → 3 × 8 = **24 bit/piksel**; 2^{24} = **16,777,216** renk. *(2 puan.)* (c) 100 × 100 × 24 bit = 240,000 bit ÷ 8 = **30,000 bayt** (ya da 10,000 × 3 bayt). *(3 puan.)*

Puanlama: 85–100 sınava hazırsın; 70–84 yanlış yaptığın konuların "Sık yapılan hatalar" kutularına dön; 70'in altı ilgili bölümün Kendin çöz sorularını tekrar çöz.

Terim Sözlüğü (İngilizce–Türkçe)

İngilizce	Türkçe	Tanım	Bölüm
address	adres	Bellekteki bir konumu benzersiz tanımlayan numara	6
address space	adres uzayı	n adres bitiyle ulaşılabilen 2^n adresin tamamı	6
analog signal	analog sinyal	Zamanla sürekli değişen sinyal (ör. mikrofon çıkışı)	5
ASCII	ASCII	7 bitlik, 128 kodlu İngilizce karakter kodlaması	3
base (radix)	taban	Konumsal sistemde rakam sayısı ve ağırlıkların temeli	2
binary	ikilik (ikili) sistem	Taban 2; rakamlar 0 ve 1	2
bit (binary digit)	bit	En küçük sayısal veri birimi, 0 ya da 1	1
bit depth	bit derinliği	Ses örneği başına bit; 2^b nicemleme seviyesi verir	5
bit pattern	bit deseni	Bitlerden oluşan dizi; uzunluğu bit sayısıdır	1
bitmap (raster) image	bit eşlem (raster) görüntü	Piksel matrisinden oluşan görüntü	4
byte	bayt	8 bit	1
byte-addressable	bayt adreslemeli	Her adresin bir baytı gösterdiği bellek	6
ceiling	tavan fonksiyonu	Bir sayıyı yukarı en yakın tam sayıya yuvarlama, $\lceil x \rceil$	6
channel	kanal	RGB'de R, G veya B bileşeni; seste mono/stereo yolu	4
code page	kod sayfası	8 bitlik kodlama; 128–255 aralığı dile göre değişir	3

İngilizce	Türkçe	Tanım	Bölüm
code point	kod noktası	Unicode'da bir karakterin kimlik numarası (U+XXXX)	3
code unit	kod birimi	Bir UTF kodlamasının temel bit grubu (8/16/32 bit)	3
color depth (bpp)	renk derinliği	Bir piksel için kullanılan bit sayısı	4
context	bağlam	Veriyi yorumlanabilir kılan çevre bilgisi	1
control character	kontrol karakteri	Görünür şekli olmayan, bir işi tetikleyen ASCII kodu	3
data	veri	Ham gözlem, ölçüm veya semboller	1
decimal	onluk sistem	Taban 10; rakamlar 0–9	2
decoding	kod çözme	Bit dizisini kullanıcıya uygun biçime çevirme	1
digitization	sayısallaştırma	Fiziksel sinyali sayılara dönüştürme	1
EBCDIC	EBCDIC	IBM mainframe'lerinin 8 bitlik, ASCII ile uyumsuz kodlaması	3
encoding (coding)	kodlama	Veriyi bit dizisi olarak temsil etme kuralı	1
endianness	bayt sırası	Çok baytlı sayının baytlarının bellekteki sırası	6
frame	kare	Videoyu oluşturan tek görüntü	5
frame rate (fps)	kare hızı	Saniyede gösterilen kare sayısı	5
glyph	glif	Bir karakterin yazı tipindeki çizimi	3
grayscale	gri tonlama	Pikselin yalnızca parlaklık seviyesi sakladığı görüntü	4
hexadecimal	onaltılık sistem	Taban 16; rakamlar 0–9, A–F	2
information	bilgi	Bir soruyu cevaplamak için yorumlanmış veri	1
memory cell	bellek hücresi	Belleğin adreslenebilen en küçük bölmesi	6
metadata	üst veri	Veri hakkında veri (süre, çözünürlük, tarih...)	5

İngilizce	Türkçe	Tanım	Bölüm
noise margin	gürültü payı	İki mantık seviyesi arasındaki güvenlik aralığı	2
octal	sekizlik sistem	Taban 8; rakamlar 0–7	2
palette (indexed color)	palet (dizinli renk)	Pikselin renk yerine renk tablosundaki sırayı sakladığı yöntem	4
pixel	piksel	Bit eşlem görüntünün en küçük renkli noktası	4
place value (weight)	basamak değeri (ağırlık)	Bir konumun çarpanı, b^i	2
positional number system	konumsal sayı sistemi	Rakamın değerinin konumuna da bağlı olduğu sistem	2
prefix	önek	Tabanı belirten ek (0x)	2
quantization	nicemleme	Ölçülen değeri sonlu seviyelerden birine eşleme	5
remainder	kalan	Bölme işleminden artan; dönüşümde rakamları verir	2
resolution	çözünürlük	Yatay × dikey piksel sayısı	4
RGB	RGB	Kırmızı, yeşil, mavi ışıkla renk oluşturma modeli	4
sampling	örnekleme	Sinyali belirli anlarda ölçme	5
sampling rate	örnekleme hızı	Saniyedeki örnek sayısı (Hz)	5
uncompressed	sıkıştırılmamış	Ham, küçültülmemiş veri boyutu	4
Unicode	Unicode	Dünyadaki karakterlere tek kod noktası veren standart	3
UTF-8 / UTF-16 / UTF-32	UTF-8 / 16 / 32	Unicode kod noktalarını 1–4 / 2–4 / 4 baytla saklayan kodlamalar	3
vector graphics	vektör grafik	Geometrik nesnelerle tanımlanan, ölçeklenebilir görüntü	4

Hocaya Sorulacak Noktalar

1. **Unicode kaç bit? (D1 s. 37)** Slayt "16 bit, 65,536 kod, dünyanın bütün dilleri için yeterli" diyor. Güncel Unicode U+10FFFF'e kadar (1,114,112 kod noktası) genişliyor ve slayt s. 41'deki tablo da UTF-16'nın 4 bayt kullanabildiğini söylüyor. Sınavda "Unicode kaç bit kullanır?" sorulursa beklenen cevap 16 bit mi?
2. **"Microsoft uses the first 256 symbols" (D1 s. 37)** ifadesi neyi kastediyor? Windows'un dahili olarak UTF-16 kullandığı biliniyor.
3. **Siyah piksel 0 mı 1 mi? (D1 s. 44 ve s. 45)** s. 44 "0 siyah, 1 beyaz" diyor; s. 45'teki şekilde siyah kareler 1 ile gösterilmiş. Sınavda hangi kural esas alınacak?
4. **Formül gösterimi (D1 s. 14):** alt indiste β, toplamda b kullanılmış; ikisi de taban mı? Toplamın sınırları (i = 0...k) yazılmalı mı?
5. **D1 s. 38'deki tabloda "OE09"** O harfiyle yazılmış; doğrusu U+0E09 (sıfır) olmalı.
6. **RGB tablosu (D1 s. 49):** (157, 95, 82) "purple" olarak etiketlenmiş ama ekranda kırmızımsı kahverengi görünüyor. Renk adları sınavda sorulur mu?
7. **"Extended ASCII" (D1 s. 35):** gösterilen tablo IBM Code Page 437. Sınavda "extended ASCII" tek bir standart gibi mi ele alınacak?
8. **Değişken yerleşimi (D1 s. 62):** 0107 adresinin neden boş bırakıldığı (hizalama) ve b = 0A 21 3A 51'in hangi sayıya karşılık geldiği (bayt sırası) sınav kapsamında mı?
9. **Ses boyutu:** Slaytlarda ses dosyası boyutu için formül yok (s. 59), ama s. 2 ve s. 66 ses boyutundan söz ediyor. Ses boyutu hesabı sorulacak mı?
10. **Eksik slaytlar:** Slayt numaraları 30–31, 42–44, 58, 61, 63–66 ve 72–75'te atlıyor; s. 34 ve s. 35 aynı numarayı (35) taşıyor. Sunumdan çıkarılan slaytlar sınav kapsamında mı?
11. **Birimler:** Slaytlar MiB/GiB kullanıyor. Sınavda MB/GB (10'un kuvvetleri) ile cevap kabul ediliyor mu, sonuçlar kaç ondalığa yuvarlanmalı?

Kaynak Kapsamı Kontrol Tablosu

Kaynak başlığı	D1 sayfa (slayt)	Bu kitapta	Durum
Kapak, haftanın soruları, öğrenme çıktıları	1–3	Başlangıç; her bölümün öncelik gerekçesi	Tam
Veri ve bilgi; sınıflandırma etkinliği	4–5	1.1, Örnek 1.1	Tam
Veri türleri	6	1.2	Tam
Kodlama/kod çözme; fiziksel dünyadan bitlere	7–8	1.3	Tam
Bit deseni	9	1.4	Tam
Bit sayısı ve olası değerler	10	1.5	Tam
Bölüm 01 kapağı	11	Bölüm 2 girişi	Tam
Konumsal sistem, 3125 örneği	12–13	2.1	Tam
Taban gösterimi	14	2.2	Tam (gösterim tutarsızlığı notlandı)
İkilik basamaklar, donanım avantajı	15–16	2.3	Tam
Tabanlar tablosu	17	2.4	Tam
Onaltılık: 3AB, A–F, 4 bit, kullanım, 0x	18–21, 23	2.5	Tam
Dönüşümler: CA52, 125 → ikilik, 125 → 7D, alıştırma, stratejiler	22, 24–27	2.6	Tam
On dönüşüm alıştırması ve cevapları	28–29	2.7	Tam (ara adımlar eklendi)
Bölüm 02 kapağı	30	Bölüm 3 girişi	Tam
BYTE kelimesi	31 (32)	3.1	Tam
ASCII, ASCII tablosu	32–33 (33–34)	3.2	Tam
8 bit kod sayfaları, genişletilmiş ASCII tablosu	34–35 (35, 35)	3.3	Tam
EBCDIC	36 (36)	3.4	Tam
Unicode; örnek karakterler; kod noktası	37–39 (37, 41, 38)	3.5	Tam (s. 37 düzeltildi)

Kaynak başlığı	D1 sayfa (slayt)	Bu kitapta	Durum
UTF-8; UTF-8/16/32 tablosu	40–41 (39–40)	3.6	Tam
Bölüm 03 kapağı	42	Bölüm 4 girişi	Tam
Piksel matrisi	43 (45)	4.1	Tam
Bir bitlik pikseller; doğrusal saklama	44–45 (46–47)	4.2	Tam (0/1 çelişkisi notlandı)
Gri tonlama; renk derinliği	46–47 (48–49)	4.3	Tam
RGB modeli ve değerleri	48–49 (50–51)	4.4	Tam
Çözünürlük, Full HD boyutu, 800 × 600 boyutu	50–52 (52–54)	4.5	Tam
Palet	53 (55)	4.6	Tam
Vektör grafik; bitmap–vektör	54–55 (56–57)	4.7	Tam
Bölüm 04 kapağı	56	Bölüm 5 girişi	Tam
Video kareleri; sıkıştırılmamış video	57–58 (59–60)	5.1	Tam
Analogdan sayısal sese	59 (62)	5.2	Tam (boyut formülü kaynak dışı eklendi)
Bölüm 05 kapağı	60	Bölüm 6 girişi	Tam
Bellek hücreleri	61 (67)	6.1	Tam
Bayt adreslemeli bellek	62 (68)	6.2	Tam (+ C ile doğrulama)
Adres biti sayısı; kapasite tablosu	63–64 (69–70)	6.3	Tam
Onaltılık adresler	65 (71)	6.4	Tam
Haftanın özeti	66 (76)	Hızlı tekrar, bağlantılar	Tam

Sayfa numarası notları: Etiketlerde fiziksel PDF sayfası kullanıldı. 30. sayfadan sonra slayt numaraları fiziksel sayfalardan farklı (parantez içinde). Slayt numaralarında boşluklar var (30–31, 42–44, 58, 61, 63–66, 72–75); bu slaytlar PDF'te yok, dolayısıyla işlenemedi.

Doğrulama: Slaytlardaki bütün hesaplar ve bu kitaptaki bütün sayısal örnekler Python ile kontrol edildi. Tek kod örneği (Bölüm 6, `adresler.c`) gcc -Wall -Wextra ile uyarısız derlenip çalıştırıldı; adres değerleri çalıştırmaya göre değişir, 4 baytlık fark değişmez.